AUTOFORGE: A CHECKPOINTED, HUMAN–IN–THE–LOOP MULTI-AGENT SDLC PIPELINE

INDIVIDUAL THESIS

SANTHUKA D.N.M.D

IT22098078

Individual component: Requirement Generation, Domain Enrichment and Checkpointed Orchestration

Group ID: R26-SE-033

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Software Engineering

Department of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Santhuka D.N.M.D

Student ID: IT22098078

Signature:  Date: 05/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: **Prof. Nuwan Kodagoda**		
Co-supervisor: **Mr. Eishan Weerasinghe**		10/10/2026

ABSTRACT

Multi-agent systems that build software with large language models usually start from a requirement the model has to guess at, and they review that requirement, if at all, through agent dialogue. A wrong early assumption then travels into the design and the code. This thesis covers the first two stages of AutoForge, a seven-agent pipeline for e-commerce web development, and the orchestration layer that joins all seven. The Requirement Agent fills gaps through a short conversation, asking at most three questions per turn, and a deterministic quality gate blocks confirmation while key fields are vague or assumed. The SRS is then generated through a single schema-constrained call, with a JSON repair step and a deterministic fallback. The Domain Agent retrieves domain knowledge from a ChromaDB index and asks the model only for a small, cited enrichment plan, which Python validates and merges, so the model never retypes the specification. A LangGraph state graph checkpointed to MongoDB pauses at every human gate and resumes after any delay or restart. Following a design-science approach, I evaluated the work within the team's three-feature case study. With retrieval enabled, useful domain requirements rose from 8 to 14, missing requirements fell from 7 to 2 and human revisions halved. All seven controlled interruptions resumed at the correct stage with earlier approvals intact. Human review caught a missing pair of Admin and Manager user stories before any design existed. A real run on a retail store feature also exposed two weaknesses that only a reviewer caught: an irrelevant guest-checkout constraint pulled in by retrieval, and a citation check that a reviewer's question could satisfy. The results suggest that grounding and gating the earliest stages is a practical way to stop specification errors from reaching code, provided the review step stays meaningful.

Keywords: requirements engineering, retrieval-augmented generation, human-in-the-loop, checkpointed orchestration, multi-agent systems

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Prof.Nuwan Kodagoda, and my co-supervisor, Mr.Eishan Weerasinghe, for their guidance throughout this project. Their insistence that every claim about the pipeline be backed by a measurement, not a description, changed how I evaluated my own component and is the reason Chapter 6 reports what went wrong as well as what worked.

I am grateful to my teammates, Sansala T.G.B.D., Dissanayake T.C. and Oberathna R.D.T.D. Their agents were the first real consumers of the specifications my component produces, and several of the design changes described in this thesis started as a bug they ran into downstream.

I also thank the academic staff of the Department Computing at the Sri Lanka Institute of Information Technology, and the panel members at the progress presentations, whose questions helped narrow the scope of this work. Finally, I thank my family and friends for their patience over the many long evenings this project took.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 3
1.4 Research gap 4
1.5 Aim and objectives 4
1.6 Research questions or hypotheses 5
1.7 Scope limitations and assumptions 5
1.8 Individual contribution 6
1.9 Thesis structure 6
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review method and scope 8
2.2 Domain and theoretical foundations 8
2.2.1 Requirements specifications as the root of the pipeline 8
2.2.2 Retrieval-augmented generation 9
2.2.3 Traceability into design 9
2.2.4 Human oversight of agent plans 9
2.2.5 Design science 10
2.3 Related research 10
2.4 Existing systems products and techniques 11
2.5 Gap synthesis and conceptual framework 12
3 RESEARCH METHODOLOGY 13
3.1 Research design 13
3.2 Data participants or study materials 13

3.3 Experimental or study procedure 14
3.4 Measures and evaluation metrics 15
3.5 Data analysis 16
3.6 Ethics privacy security and responsible AI 16
3.7 Reproducibility and research artefacts 17
4 PRODUCT AND SOLUTION DESIGN 18
4.1 Requirements 18
4.2 Component architecture within the complete solution 19
4.3 Design alternatives and decisions 20
4.4 Detailed design 22
4.4.1 Conversational requirement elicitation 22
4.4.2 SRS structure and generation 23
4.4.3 Revision through targeted operations 24
4.4.4 Knowledge ingestion and retrieval 25
4.4.5 Enrichment plan, validation and merge 26
4.4.6 Orchestration as a checkpointed state graph 27
4.4.7 Approval and versioning 28
4.5 Standards and professional practice 28
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development environment and technology stack 30
5.2 Component implementation 30
5.2.1 Requirement Agent 31
5.2.2 Retrieval pipeline 32
5.2.3 Domain Agent 32
5.2.4 Orchestrator and approval service 33
5.2.5 LLM provider service 33

5.3 Integration with the other components 34
5.4 Verification and software testing 34
5.5 Deployment and operation 35
6 RESULTS AND VALIDATION 37
6.1 Research results 37
6.2 Product and technical results 38
6.2.1 Effect of retrieval on completeness (RQ1) 38
6.2.2 Small plan versus full regeneration (RQ2) 39
6.2.3 Checkpoint recovery (RQ3) 39
6.2.4 Human review at the design stages (RQ4) 40
6.3 User or stakeholder validation 41
6.4 Worked Example: Retail Store "Add & List Items" 41
6.4.1 The generated SRS 41
6.4.2 The Enhanced SRS and its revisions 42
6.5 Integration and Performance Results 44
6.6 Summary of Findings 45
7 DISCUSSIONS 46
7.1 Interpretation of findings 46
7.2 Comparison with literature baselines and existing solutions 47
7.3 Implications for Practice 47
7.4 Threats to Validity 48
7.5 Limitations 48
8 COMMERCIALISATION AND SUSTAINABILITY 50
8.1 Value of the Component 50
8.2 Target Users and Market Position 51
8.3 Business Model and Feasibility 53

8.4 Sustainability and Maintenance 55
8.5 Contribution to the Sustainable Development Goals 56
9 CONCLUSION AND FUTURE DIRECTIONS 57
9.1 Conclusion 57
9.2 Achievement of objectives 57
9.3 Contribution 58
9.4 Future directions 58
REFERENCES 60
GLOSSARY 63
APPENDICES 64
Appendix A Supporting Technical Material 64
A.1 Test catalogue 64
A.2 Quality gate rules 65
A.3 Domain Improvements report, retail store version 1 65
Appendix B: Disclosure of AI Tool Use 66

LIST OF FIGURES

Figure 1.1: The AutoForge pipeline System Diagram 2
Figure 2.1: Conceptual framework for the requirement, domain and orchestration . 12
Figure 4.1: The component's place in the complete AutoForge architecture 20
Figure 4.2: Requirement elicitation, quality gate and SRS generation 22
Figure 4.3: Domain knowledge ingestion and the enrichment pipeline 25
Figure 4.4: The orchestration graph with five hard approvals and two soft gates 27
Figure 6.1: Retrieval ON versus OFF on the same request and model 38

LIST OF TABLES

Table 2.1: Requirement-stage properties of related systems 11
Table 3.1: Domain knowledge base used in the evaluation 14
Table 3.2: Measures used for each research question 15
Table 3.3: Configuration parameters relevant to reproducing the results 17
Table 4.1: Functional requirements of the component 18
Table 4.2: Non-functional requirements of the component 19
Table 4.3: Main design decisions 20
Table 4.4: Sections of the generated SRS 23
Table 5.1: Technology stack used by the component 30
Table 5.2: Main modules of the component 31
Table 5.3: Automated tests for the component 34
Table 6.1: Key behaviours confirmed by automated tests 37
Table 6.2: Retrieval ablation on one controlled request 38
Table 6.3: Observed behaviour of the two Domain Agent designs 39
Table 6.4: Controlled checkpoint-recovery trials 40
Table 6.5: Pipeline with human gates on and off 40
Table 6.6: Composition of the retail store SRS, version 1 42
Table 6.7: Enhanced SRS versions for the retail store feature 43
Table 6.8: Execution performance of the full pipeline 44
Table 6.9: Summary of findings by research question 45
Table 8.1: Target users of the component 50
Table 8.2: Alternatives for producing a feature specification 52
Table 8.3: Revenue elements that depend on this component 53
Table 8.4: Work needed before a commercial release 54
Table 9.1: Achievement of the objectives 57
Table A.1: Test files for the component 64
Table A.2: Rules applied by the requirement quality gate 65
Table B.1: Use of generative AI tools 66

IST OF ABBREVIATIONS

Abbreviation	Meaning
AC	Acceptance Criterion
AI	Artificial Intelligence
API	Application Programming Interface
BA	Business Analyst
CRUD	Create, Read, Update and Delete
DOCX	Office Open XML Word Document
FR	Functional Requirement
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
PDF	Portable Document Format
QA	Quality Assurance
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RQ	Research Question
SDG	Sustainable Development Goal
SDLC	Software Development Life Cycle
SME	Small and Medium-sized Enterprise
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language

1 INTRODUCTION

1.1 Overall project context

AutoForge is our team's final-year research project. It is a multi-agent system that builds features of an e-commerce web application with large language models (LLMs), but it does so in stages, the way a software team would, rather than jumping from a prompt to finished code. Seven agents take turns: a Requirement Agent writes the Software Requirements Specification (SRS), a Domain Agent enriches it with domain knowledge, an Architecture Agent designs the system, a UI/UX Agent designs the screens, a Coder Agent implements the change, and Security and Quality Assurance (QA) agents check the result. Each agent produces a versioned artefact, and the pipeline pauses so that a person can approve it, reject it or ask for changes before the next agent sees it.

An LLM can already write a working function or a page on request, but a full application depends on a chain of decisions, and tools that generate everything from one prompt leave no specification to check and no way to tell whether a poor result came from a misunderstood requirement or a coding mistake. Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] improve on this by giving agents separate roles, but review still happens through agent conversation or after the run ends. Figure 1.1 shows the AutoForge pipeline.

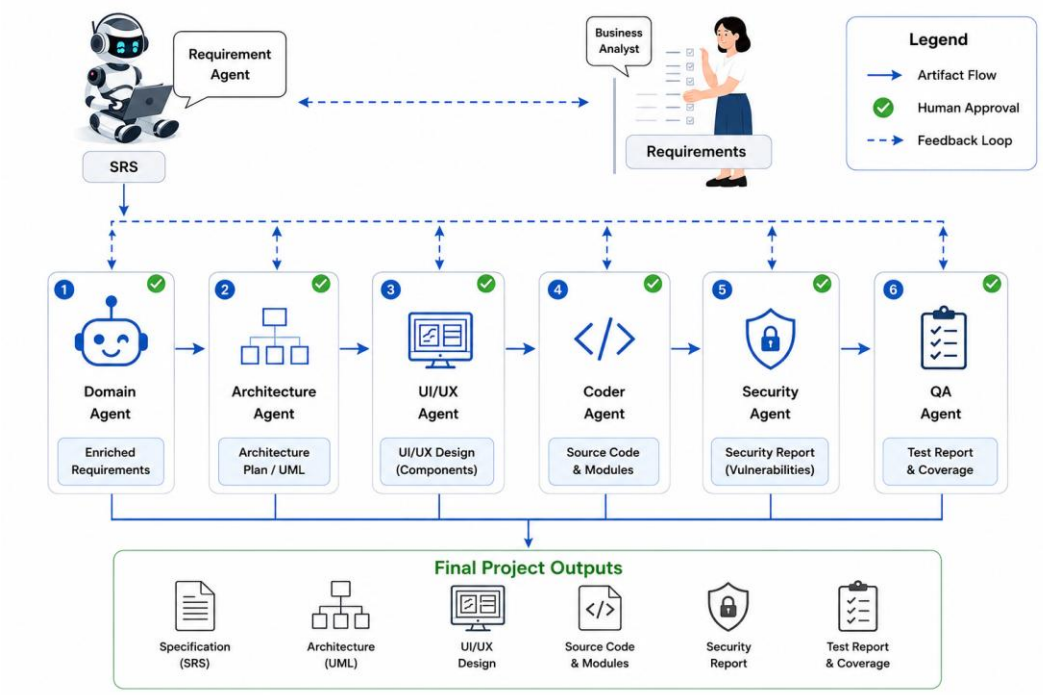


Figure 1.1: The AutoForge pipeline System Diagram

The project was divided into four components, one per team member. My component covers the first two stages of the pipeline, the Requirement Agent and the Domain Agent, together with the orchestration layer that connects all seven agents, pauses them for human decisions and lets the pipeline survive interruptions. The other three components cover architecture and UI/UX design, code generation, and security and quality assurance. The team's common report describes the integrated system; this thesis describes my component in detail.

1.2 Individual component context

The first two stages of a development pipeline matter more than their size suggest. Every later agent builds on the specification: the Architecture Agent derives its plan and diagrams from it, the UI/UX Agent decides which screens to draw from it, and the Coder Agent decides what to implement from it. A gap or a wrong assumption at this point does not stay small. In e-commerce, where features touch accounts, stock levels, orders and payment data, a requirement that is cheap to fix while it is still a sentence in a specification becomes expensive once database models, endpoints and pages have been built around it.

My component therefore has three jobs. The first is to turn a rough feature idea into a specification that is complete enough for the downstream agents to use without guessing. The second is to check that specification against domain knowledge, so that conventions a general-purpose model might miss, such as how inventory or order states are usually handled, are added and clearly marked. The third is to make the whole pipeline controllable: every stage has to be able to wait for a person, for minutes or days, without losing its place, and to continue correctly once a decision arrives.

These three jobs are connected. A specification is only worth reviewing if the review can actually stop the pipeline, and a pause is only practical if the system can be restarted while it waits. That is why I treat the early agents and the orchestration layer as one component rather than three separate pieces.

1.3 Problem statement

When an LLM-based system writes a requirements specification for a new feature, it usually works from whatever the user typed and from the model's own general knowledge. Three problems follow. First, the input is often incomplete, and the model fills the gaps silently, so the specification looks complete while resting on assumptions nobody agreed to. Second, the model's general knowledge may not include the conventions of the target domain, and when it does try to add them, there is no way to tell which additions are grounded in a real source and which were invented. Third, asking a model to regenerate a whole document to make a small change is unreliable: during our development, three different locally hosted models dropped required sections, omitted keys or produced truncated output when asked to retype the full specification.

On top of this, existing multi-agent frameworks do not give a person a durable point of control. Review happens through agent dialogue or after the run, and an interrupted run is generally lost. The problem this thesis addresses is therefore: how can the requirement and domain stages of an LLM-based development pipeline produce a specification that is complete, grounded in cited domain knowledge, safely

revisable and approved by a person, within a pipeline that can pause indefinitely and resume correctly after interruption?

1.4 Research gap

The literature reviewed in Chapter 2 points to a gap with four parts. Role-based systems such as MetaGPT [2] produce an SRS-like document, but they build it from the model's parametric knowledge without retrieving outside domain knowledge in the sense Lewis et al. [3] established, and Hemmat et al. [4] describe domain grounding of LLMs in requirements engineering as an under-explored direction. None of the systems we reviewed treats the specification as an independently versioned artefact with its own accept, reject or revise decision, even though Takerngsaksiri et al. [5] found that engineers value oversight at the planning stage as much as at the code stage. Traceability research [6], [7] shows that most solutions cover one narrow link rather than a chain from requirement to code, which needs stable identifiers from the very first artefact. Finally, none of these systems reports a pipeline state that survives a restart, which is what makes a human gate practical.

What is missing, then, is a requirement stage that asks before it assumes, a domain stage that can only add what it can cite, a revision mechanism that changes exactly what was asked and nothing else, and an orchestration layer that holds the whole pipeline still while a person decides.

1.5 Aim and objectives

The aim of this work is to design, implement and evaluate the requirement and domain stages of AutoForge, and the checkpointed orchestration layer that controls the full pipeline, so that the specification every later agent depends on is complete, grounded, traceable and human approved.

To achieve this aim I set four objectives:

1. O1: Build a Requirement Agent that fills gaps through a short conversation, refuses to confirm a vague specification without an explicit override, and produces a schema-valid SRS with stable identifiers and traceability.

2. O2: Build a retrieval pipeline and a Domain Agent that enrich the approved SRS only with knowledge they can cite, without asking the model to regenerate the specification.
3. O3: Build an orchestration layer that pauses the pipeline at human approval gates, records each decision, and resumes from the exact paused point after any delay or restart.
4. O4: Evaluate the effect of retrieval on specification completeness, the reliability of checkpoint recovery, and the value of human review at the early stages.

1.6 Research questions or hypotheses

The objectives lead to four research questions. They are narrower versions of the pipeline-level questions in our research paper, focused on the part of the system I built.

1. RQ1: Does retrieval-grounded enrichment produce a more complete specification than generation without retrieval?
2. RQ2: Does a "small plan, deterministic merge" design preserve the specification's structure and identifiers better than asking the model to regenerate the whole document?
3. RQ3: Can a checkpointed state graph hold the pipeline at a human gate and resume it correctly after an interruption?
4. RQ4: Does human review at the requirement and domain stages catch problems before they reach design and code?

1.7 Scope limitations and assumptions

The work covers the requirement and domain stages for individual features of e-commerce web applications, and the orchestration of all seven stages. The domain knowledge base used in the evaluation is an e-commerce knowledge base of five short documents covering catalogue and inventory, carts and checkout, order life cycles, payments and accounts. The design itself is domain-independent: neither the validator nor the merge logic contains e-commerce rules, so another knowledge base could be swapped in.

The main limitations follow from the evaluation scale. Retrieval was compared with and without enrichment on one controlled request, checkpoint recovery was tested with seven controlled interruptions, and the case study covers three features. These are enough to show that the mechanisms work and to measure their effect, but not to give general rates with statistical confidence. The knowledge base is also small, which limits what retrieval can contribute and, as Chapter 6 shows, makes irrelevant matches more likely.

I assume that a person with enough knowledge of the feature is available to answer the requirement questions and review each artefact, though not necessarily straight away, since the pipeline is designed to wait. I also assume that the machine running AutoForge can host the default model through Ollama and reach a MongoDB database for artefacts and checkpoints.

1.8 Individual contribution

My contributions to the project, all of which are reflected in the shared repository's history, are:

- the Requirement Agent, including the conversational gap-filling loop, the deterministic quality gate, SRS generation with a repair-and-fallback ladder, a completeness backstop, and revision through targeted operations;
- the domain knowledge pipeline (document loading, chunking, embedding and ChromaDB storage) and the Domain Agent's plan, validate and merge design;
- the LangGraph orchestration layer with its MongoDB checkpointer, and the approval service that turns a person's decision into a resumed pipeline;
- the per-agent LLM provider service that lets each agent run on a different model or provider; and
- the automated tests for these modules, and my share of the team's evaluation

1.9 Thesis structure

Chapter 2 reviews the literature and existing systems and builds a conceptual framework. Chapter 3 describes the research method. Chapter 4 presents the design of the component and the decisions behind it, and Chapter 5 covers implementation,

integration with the other components, and testing. Chapter 6 reports the results, including a real run on a retail store feature, and Chapter 7 discusses what they mean. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

I started from two broad surveys, Hou et al.'s systematic review of LLMs for software engineering [8] and He, Treude and Lo's review of LLM-based multi-agent systems for software engineering [9], and followed their references to the systems most relevant to AutoForge. From there the review narrowed to four themes that bear directly on my component: multi-agent systems that produce specifications, human oversight of agents, retrieval-augmented generation, and requirements traceability. Work on testing and security with LLMs [10], [11] and on coding agents [12], [13] was included as context, since the specification my component produces is what those later stages build on.

I included work that describes a system or method concretely enough to compare with ours, and surveys that summarize a research area. I did not attempt a full systematic review. The aim was to locate the gap precisely enough to design against it.

2.2 Domain and theoretical foundations

2.2.1 Requirements specifications as the root of the pipeline

A requirements specification records what a system must do and under what conditions, in a form that both people and later development stages can use. In AutoForge the SRS is not just a document for reading; it is the input every later agent consumes. That places two demands on it that a human-written specification does not always meet. It has to be structured, so that an agent can read particular sections reliably, and each requirement needs a stable identifier, so that later artefacts can point back to it. Hemmat et al. [4], reviewing research on LLMs in requirements engineering, note that work so far has concentrated on generating and classifying requirements, with less attention to grounding them in domain knowledge or to keeping a human in control of the result.

2.2.2 Retrieval-augmented generation

Lewis et al. [3] introduced retrieval-augmented generation (RAG), in which a model first retrieves passages from an external knowledge source and then generates its answer with those passages in its context. The appeal for requirements work is that the knowledge can be chosen and updated by the people using the system, rather than being fixed in the model's training data, and that each claim in the output can in principle be traced back to the passage it came from. The usual implementation embeds documents as vectors, stores them in a vector database such as Chroma [14], and retrieves the passages closest to the query by vector similarity.

Retrieval does not by itself stop a model from adding things the passages do not support, and it can retrieve passages that are similar in wording but not relevant to the task. Both problems turned out to matter in practice, and Chapter 6 reports a real case of the second.

2.2.3 Traceability into design

Gotel and Finkelstein [6] defined the requirements traceability problem as the difficulty of following the life of a requirement both backwards to its origin and forwards into design and implementation. More than three decades later, Mucha, Kaufmann and Riehle [7] report that most traceability solutions still address one narrow type of link. For an agent pipeline, traceability has a practical requirement: it can only be built if requirements carry identifiers that do not change when the document is revised. A regenerated document that renumbers its requirements breaks every link built on the previous version.

2.2.4 Human oversight of agent plans

He, Treude and Lo [9] list human-agent collaboration and the verifiability of results among the main open problems for multi-agent software engineering. Collaboration in practice means the system must be able to stop and wait for a person. Workflow frameworks such as LangGraph model an agent workflow as an explicit state graph and support pausing at a node and checkpointing the state, which Pelluru [15] discusses in the context of running multi-agent systems in production. A checkpoint that lives only in memory disappears with the process, so for a pause of hours or days the state has to be persisted outside the running application.

2.2.5 Design science

My work builds an artefact and evaluates it in its intended setting, which places it within the design-science paradigm described by Hevner et al. [16]. Design science asks that the artefact address a real problem, that it be evaluated rigorously, and that the research contribute knowledge about the design itself, not only a working system. Chapter 3 explains how I applied this.

2.3 Related research

ChatDev [1] organises role-playing agents into a chat-based "virtual software company" that follows a waterfall-like process. The agents agree on requirements and design by talking to one another. This makes the process more structured than a single prompt, but the requirements are an outcome of the conversation rather than a document a person approves before anything else happens.

MetaGPT [2] goes further towards documents. Its role agents exchange structured artefacts, including a product requirements document similar to an SRS and a design document, and each role works from the documents produced before it. This is the closest published design to the first stages of AutoForge. Two differences matter for my component. MetaGPT's specification is written from the model's own knowledge, without retrieval, and there is no persistent checkpoint at which a person can approve or reject it.

Takerngsaksiri et al. [5] describe HULA, a human-in-the-loop framework for software development agents deployed inside Atlassian JIRA. Engineers working with HULA could review the agent's plan before any code was written, and the study found that they valued this planning-stage oversight as much as code review. This is direct support for gating the requirement stage, though HULA works from issues inside an existing tracker rather than producing a new specification from scratch.

A large group of coding agents starts after the requirement stage. SWE-agent [12], OpenHands [13], CodeAct [17], HyperAgent [18] and Agentless [19] resolve tasks that are already specified, usually as GitHub issues, and are commonly evaluated on SWE-bench [20]. AgentCoder [21] adds a loop of test design and execution, and RepoCoder [22] retrieves code from the same repository to improve completion.

These systems show what an agent can do once the task is clear. They also show, by what they assume, how much depends on the task being clear in the first place, which is the job of my component.

Surveys of LLM-based testing [10] and security [11] add one more relevant point: most systems lack a deterministic verification layer and rely on the model's own judgement. The same concern applies to specifications. If the only check on a model-written SRS is another model call, nothing guarantees that a required section is present or that an identifier is unique.

2.4 Existing systems products and techniques

Table 2.1 compares the systems most relevant to the requirement stage on the properties my component is designed around. The table reports only what each publication states; N/R means the property is not reported.

Table 2.1: Requirement-stage properties of related systems

System	Specification artefact	Retrieval grounding	Human gate on specification	Durable pause
ChatDev [1]	Agreed through agent dialogue	N/R	N/R	N/R
MetaGPT [2]	SRS-like document	No (parametric)	N/R	N/R
HULA [5]	Plan from a JIRA issue	N/R	Yes (plan review)	N/R
SWE-agent / OpenHands [12], [13]	Assumes a specified issue	N/R	N/R	N/R
AutoForge (this work)	**Versioned SRS with stable IDs**	**Yes, cited**	**Yes**	**Yes (MongoDB checkpoint)**

Beyond whole systems, several techniques recur in the literature and in practice, and I drew on them directly. Constraining a model's output to a schema makes its answer machine-readable. Asking the model to repair invalid output is a common second attempt. Retrieval grounds the output in chosen sources. What is less common is the combination of these with deterministic, non-LLM steps: a rule-based check of whether a specification is concrete enough, a validator that rejects uncited additions,

and a merge performed in ordinary code so that the model never has to reproduce the whole document. Those deterministic steps are where most of my design effort went.

2.5 Gap synthesis and conceptual framework

Putting the review together, the gap my component addresses has four parts: specifications are written from incomplete input without asking, domain additions are not tied to sources, revisions risk silently changing or losing content, and there is no durable point at which a person approves the specification before the pipeline moves on.

Figure 2.1 shows the conceptual framework I used to design against this gap. It connects three inputs (the rough feature requirement, the business analyst's answers, and domain knowledge documents) through three mechanisms (conversational gap analysis, schema-constrained generation and retrieval with a small enrichment plan) and three controls (stable identifiers, citation validation and a checkpointed human gate) to three outcomes: a complete and reviewable SRS, a grounded Enhanced SRS, and a pipeline that can wait for a person and recover from interruption. The mechanisms produce content; the controls decide whether that content is allowed to move forward. Keeping these two roles separate is the central idea of the design.

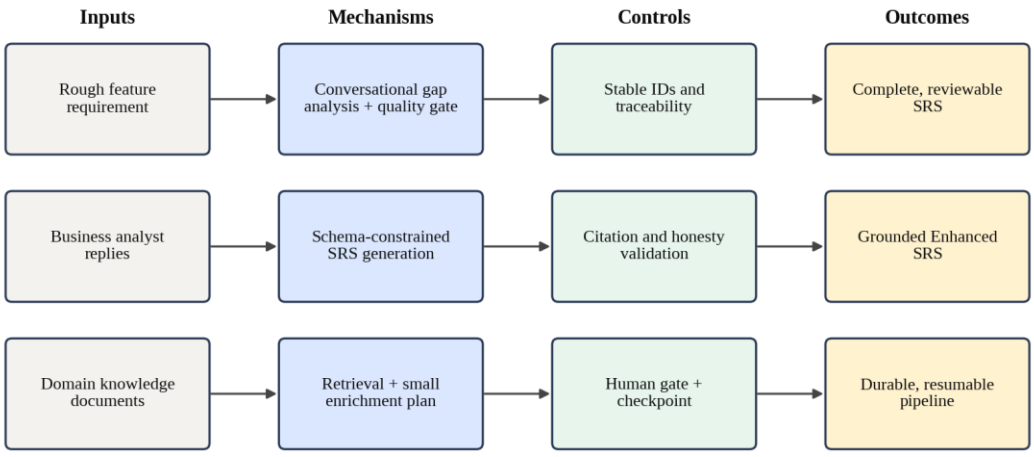


Figure 2.1: Conceptual framework for the requirement, domain and orchestration

3 RESEARCH METHODOLOGY

3.1 Research design

This is design-science research in the sense described by Hevner et al. [16]: I built an artefact to address a specific problem and evaluated it in the setting it was built for. There is no shared benchmark for going from a rough requirement to an approved, domain-grounded specification inside an agent pipeline, so a benchmark comparison was not available. Instead, the evaluation combines controlled experiments on individual mechanisms with a case study of the full pipeline on real features.

The work followed build-and-evaluate cycles across the project's milestones, between June and August 2026. Each cycle added or changed one mechanism, tested it against hand-built fixtures, and then ran it on real feature data. Several of the design decisions in Chapter 4 came directly out of these cycles. The clearest example is the Domain Agent's original design, which asked the model to return the full enhanced specification; testing showed that this failed with every model we tried, and the design was replaced. I treat these failures as findings in their own right rather than as dead ends to leave out.

Evaluation took place at two levels. At the component level, I tested the parts I built in isolation: the quality gate, the revision patcher, chunking, the plan validator, the merge logic and the state graph. At the pipeline level, my component was evaluated as part of the team's case study, which carried three features through all seven stages and included controlled experiments on retrieval, human gates and checkpoint recovery.

3.2 Data participants or study materials

The study materials were the domain knowledge base, the features used in the case study, the language models, and the artefacts the pipeline produced.

The e-commerce knowledge base contains five plain-text documents written for the project, each covering one area of e-commerce practice. Table 3.1 lists them. They are deliberately short and specific, so that a retrieved passage carries one clear

convention, such as merging a guest cart on login or locking an account after repeated failed logins.

Table 3.1: Domain knowledge base used in the evaluation

Document	Size	Main conventions covered
product_catalog_and_inventory.txt	2.6 KB	Products versus variants, SKU per variant, stock counts
checkout_and_cart_conventions.txt	2.4 KB	Guest cart merge on login, cart quantity rules
order_lifecycle_states.txt	2.4 KB	Canonical order states and allowed transitions
user_account_and_authentication.txt	2.4 KB	Account lockout after failed attempts, account rules
payment_and_pci_basics.txt	2.3 KB	Never storing raw card data, tokenisation

The case study used three e-commerce features of increasing complexity: Item Listing (a basic create, read, update and delete feature), Authentication and Authorization, and Shopping Cart. I also use a run of an "Add & List Items" feature in a retail store project as a worked example in Chapter 6, because its full artefact history, including four versions of the Enhanced SRS, is available in the project's artefact store.

The default model was qwen3-coder, served locally through Ollama. It replaced an earlier default after that model could not reliably follow the structured JSON schemas the pipeline depends on. During the Domain Agent's development, the original full-regeneration design was also tried with qwen3-coder, gemma4 and llama3.

No external human participants took part. The reviewer decisions recorded in the pipeline were made within the project, and no personal data was collected. Section 6.3 discusses what this means for validation.

3.3 Experimental or study procedure

Four procedures were used.

Retrieval ablation. To find out whether retrieval actually improves the specification, rather than assuming it does because retrieved text appears in the output, the same

requirement request was run through the domain stage twice with the same model configuration, once with retrieval enabled and once with it disabled. The two outputs were then compared on the number of requirements, the number that were useful domain requirements, the number of requirements that were missing, and the number of human revisions needed.

Checkpoint recovery trials. One real, unplanned backend failure during development showed that a paused pipeline could recover, but a single incident cannot establish a recovery rate. The team therefore introduced a controlled interruption at each of the seven stage boundaries, after the state had been saved, and judged a recovery successful only if the pipeline resumed at the correct next stage without losing the earlier approval decision.

Human-gated case study. Each feature was carried through all seven stages. At every stage the team recorded the artefact produced, what the reviewer saw, the decision taken, and any revision requested. For my component, the relevant records are the SRS and Enhanced SRS versions and the reasons for each revision. The team also ran the pipeline with human gates switched on and off to compare quality and cost.

Component testing. Throughout development, deterministic parts of the component were tested against hand-built fixtures with no model call, so the tests give the same result on every run. Parts that call a model were additionally checked by hand against real feature data at each milestone, because a response can satisfy its schema and still be wrong in substance.

3.4 Measures and evaluation metrics

Table 3.2 lists the measures used for each research question.

Table 3.2: Measures used for each research question

RQ	Measure	How it is obtained
RQ1	Requirements generated; useful domain requirements; missing requirements; human revisions	Retrieval ON/OFF comparison on the same request and model
RQ2	Sections and identifiers preserved; items added or changed; failure modes observed	Fixture tests of the merge; inspection of real Enhanced SRS versions; earlier full-regeneration attempts

RQ	Measure	How it is obtained
RQ3	Correct resume per interrupted stage; recovery rate	Seven controlled interruptions; real failure; orchestrator tests
RQ4	Revisions requested at the early stages and their causes; quality difference with gates on and off	Case study records; human-gates comparison
Cost	Total time, LLM calls, retries, tokens per run	Execution logs

3.5 Data analysis

The analysis is descriptive. With one ablation request, seven recovery trials and three features, there is not enough data for statistical tests, and I have avoided presenting the results as if there were. Counts and rates are reported directly and compared between conditions, and every number is accompanied by what it can and cannot show. Qualitative evidence, such as the content of a revision request or an addition the Domain Agent should not have made, is reported alongside the numbers because it often explains them.

3.6 Ethics privacy security and responsible AI

Several design choices in my component follow from responsible-AI concerns rather than from function alone. The default model runs locally, so a team's requirements and knowledge documents do not have to be sent to an external service. Uploaded knowledge documents are restricted to PDF, Word, text and Markdown files of up to 20 MB, and a document attached to a requirement conversation is read once for its text and then discarded rather than stored.

The more important concern is honesty about what the model knows. The Requirement Agent never presents an assumed value as if it had been given: every field it fills by default is recorded in the specification's assumptions, and a person who confirms a specification despite the quality gate must give a reason, which is written into the document. The Domain Agent may only add content it can cite, either from a retrieved passage or from the reviewer's own comment, and it may not claim domain additions when nothing was retrieved. These rules exist so that a reviewer can tell grounded content from guesses.

Finally, no stage can move the pipeline forward on its own judgement at the gated stages. A person makes every approval decision, and the system records who approved what. Chapter 7 returns to the limits of this, in particular the risk that reviewers approve without reading carefully.

3.7 Reproducibility and research artefacts

All of the component's code, prompts, schemas and tests are in the team's shared repository. The parameters that affect results are held in one configuration module and are listed in Table 3.3. Every artefact the pipeline produces is saved as a numbered version in JSON and Markdown, and as a PDF for the specifications, so the outputs reported in Chapter 6 can be inspected directly.

Table 3.3: Configuration parameters relevant to reproducing the results

Parameter	Value
Default LLM (one-shot calls)	qwen3-coder via Ollama
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Chunk size / overlap	900 / 150 characters
Retrieved chunks per query (top-k)	6
Retrieval query length cap	3,000 characters
Pinned document caps	6 chunks per document, 12 in total, 16 retrieved overall
Questions per conversation turn	At most 3
Attached document cap	8,000 characters
Vector store	ChromaDB persistent collection "domain_knowledge"
Checkpoint store	MongoDB collections langgraph_checkpoints and langgraph_checkpoint_writes

Reproducing the pipeline-level results requires a MongoDB database, a local Ollama server with the default model, and the knowledge base ingested into ChromaDB. Because model output is not fully deterministic, a repeated run will not produce word-for-word identical artefacts; the structure, identifiers and validation behaviour, however, are deterministic by design.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements for my component came from three places: the team's objectives for the whole pipeline, the needs of the downstream agents that consume the specification, and problems discovered while building it. Table 4.1 lists the functional requirements and Table 4.2 the non-functional ones.

Table 4.1: Functional requirements of the component

ID	Requirement
C1-FR-01	Hold a conversation with the business analyst that asks at most three questions per turn about what is still missing
C1-FR-02	Accept replies as free text or as an attached document, and let the analyst edit an earlier reply and regenerate from that point
C1-FR-03	Check the draft specification with a deterministic quality gate before it can be confirmed, and record any override with its reason
C1-FR-04	Generate a schema-valid SRS with stable identifiers (FR, NFR, AC, VR, US) and a traceability table
C1-FR-05	Save every SRS as a numbered version in JSON, Markdown and PDF
C1-FR-06	Revise an SRS by applying only the changes the reviewer asked for, and report any request that could not be matched
C1-FR-07	Ingest domain documents (PDF, DOCX, TXT, MD) into a vector store, globally or per project
C1-FR-08	Retrieve knowledge relevant to the approved SRS, including every chunk of any document the reviewer explicitly references
C1-FR-09	Propose cited additions and enrichments, validate them, and merge them into an Enhanced SRS with each change flagged
C1-FR-10	Produce a Domain Improvements report stating what changed, why, and from which source
C1-FR-11	Run the seven stages as one state graph per feature, pausing at each gated stage until a person decides
C1-FR-12	Resume the graph at the right point when a decision arrives, including after a backend restart
C1-FR-13	Allow only one approved version of the SRS and of the Enhanced SRS at a time, and allow an approval to be revoked until the next stage has started
C1-FR-14	Let each agent use its own model and provider (Ollama, OpenAI or Anthropic)

Table 4.2: Non-functional requirements of the component

ID	Quality	Requirement
C1-NFR-01	Reliability	No single model failure may stop SRS generation; a deterministic fallback must always produce a usable document
C1-NFR-02	Recoverability	Pipeline state must survive a process restart and resume without losing approvals
C1-NFR-03	Integrity	Requirement identifiers must never be changed or invented by the model during enrichment or revision
C1-NFR-04	Honesty	Assumed values and uncited content must be visible to the reviewer, never presented as given
C1-NFR-05	Privacy	The component must work with a locally hosted model so project data can stay on the machine
C1-NFR-06	Maintainability	Each agent must be a self-contained package; the validator and merge logic must contain no domain-specific rules
C1-NFR-07	Testability	Deterministic logic must be testable without calling a model
C1-NFR-08	Responsiveness	Long model calls must stream partial output to the interface so the analyst is not left waiting blind

4.2 Component architecture within the complete solution

Figure 4.1 places my component within the complete system. The analyst and the reviewers work through a React interface, which calls the FastAPI backend. Within the backend, my component consists of the Requirement Agent, the Domain Agent, the approval service, the graph orchestrator and the LLM provider service. The Requirement and Domain agents write their artefacts to the versioned artefact store; the approval service records decisions and, when a gated artefact is approved, tells the orchestrator to resume. The orchestrator keeps its checkpoints in MongoDB, and the Domain Agent queries ChromaDB.

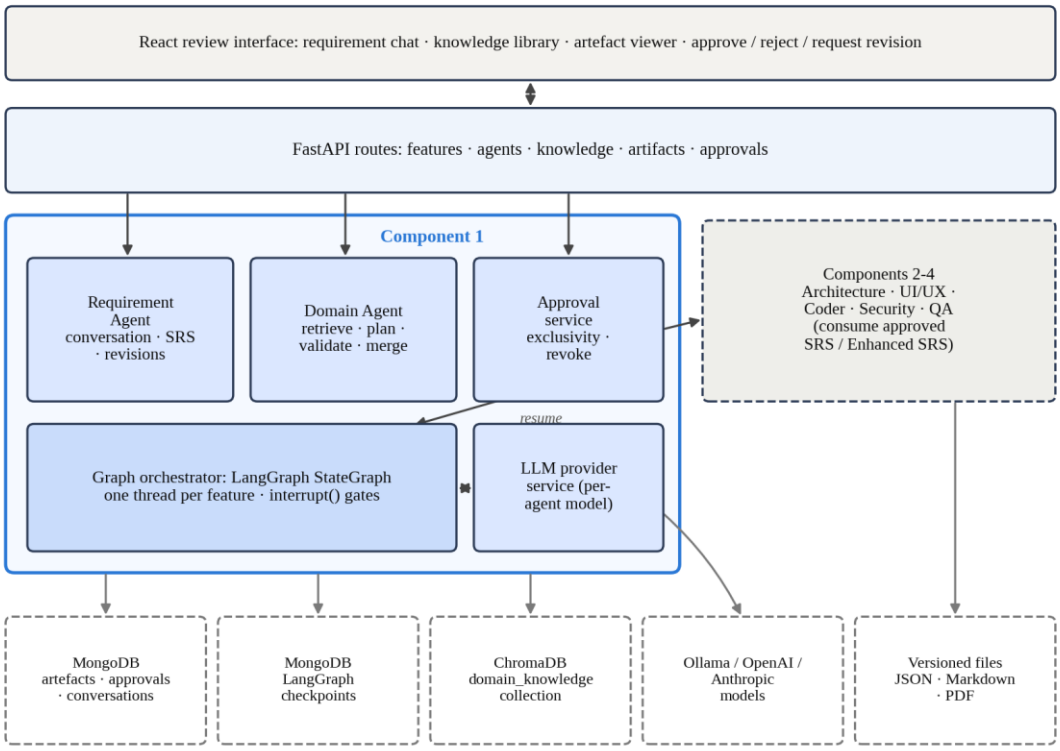


Figure 4.1: The component's place in the complete AutoForge architecture

The boundary with the other components is the approved artefact. The Architecture, UI/UX and Coder agents never call my agents directly. They read the latest approved Enhanced SRS, or the approved SRS if no Enhanced SRS has been approved, and the orchestrator runs them in order. This keeps the components loosely coupled: as long as the specification keeps its agreed shape, any agent can change its internals without breaking the others.

4.3 Design alternatives and decisions

Several decisions shaped the component. Table 4.3 summarizes them, the alternative that was considered or tried, and the reason for the choice. The most important of these are explained in more detail below.

Table 4.3: Main design decisions

Decision	Alternative	Reason for the choice
Conversation before generation	One-shot form submitted to the agent	Gaps are filled by asking, not guessing; vague fields are caught before they reach the Architecture Agent

Decision	Alternative	Reason for the choice
Deterministic quality gate	Ask the model whether the draft is ready	Gives the same answer every time and can be tested; a model is not a reliable judge of its own gaps
Small enrichment plan + Python merge	Model returns the full enhanced SRS	Full regeneration dropped sections, omitted keys or truncated output with all three models tried
Revision as targeted operations	Model retypes the whole SRS with the change	A requested change either happens or is reported as unmatched; it cannot silently disappear
Deterministic retrieval	Model decides what to retrieve through tool calls	Predictable, testable retrieval on the simpler one-shot model path
ChromaDB	MongoDB Atlas Vector Search	No extra index infrastructure at this scale; persistent local storage
Local sentence-transformer embeddings	Embeddings from the chat model provider	Retrieval does not depend on which chat provider an agent uses
LangGraph with MongoDB checkpointer	In-memory state, or calling agents in sequence by hand	Native pause and resume; state survives a restart
Approval resumes the graph	Graph polls for decisions	A decision takes effect immediately and the approval itself is the single source of truth

The decision that cost the most to reach was the move from full regeneration to a small plan. The first version of the Domain Agent asked the model to return the entire enhanced specification together with a summary of changes. Real end-to-end tests with three locally hosted models, qwen3-coder, gemma4 and llama3, showed the same failure pattern: required sections were silently dropped, top-level keys went missing, and long outputs were truncated or malformed. The problem was not the particular model but the task. Asking any model to reproduce a large document word for word, while changing only a few items, gives it many chances to make an unrequested change. The replacement asks the model only for the changes, as a short list, and leaves the copying to ordinary code that cannot make that mistake. The same reasoning later led me to redesign SRS revision in the same way.

4.4 Detailed design

4.4.1 Conversational requirement elicitation

A feature starts with a short description, often a sentence or two. Rather than generating an SRS from that directly, the Requirement Agent runs a conversation. In each turn it makes one small model call, the gap analysis, which reads what is known so far, extracts anything new from the analyst's last reply, and asks at most three questions about what is still missing. Each question comes with a realistic example answer, so the analyst can adapt an example instead of writing from scratch. The known answers and a preview of the SRS are updated after every turn and stored, one document per feature, in MongoDB. Figure 4.2 shows the flow.

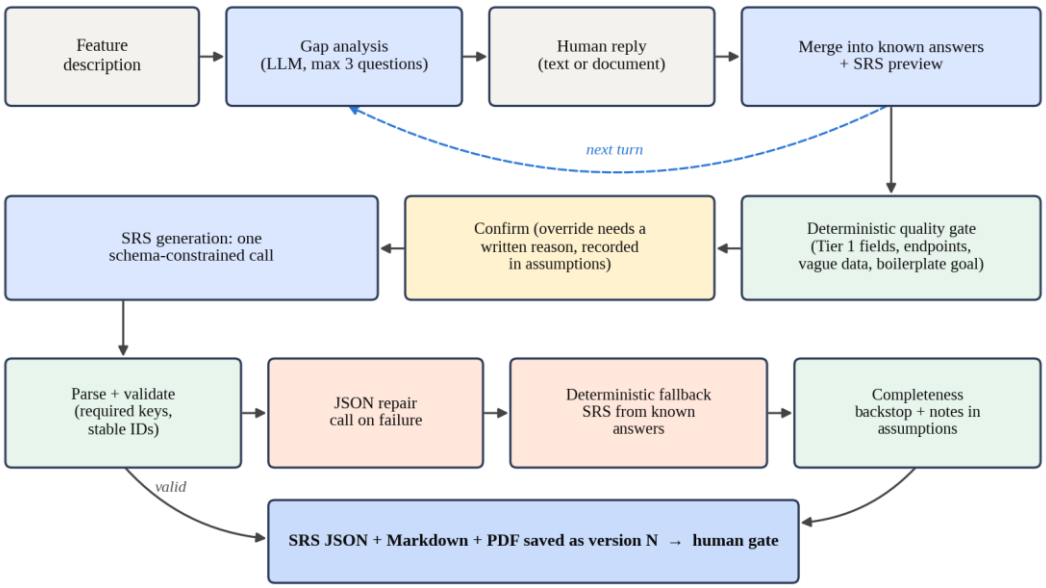


Figure 4.2: Requirement elicitation, quality gate and SRS generation

Not every field is equally important, so the questions are prioritised in three tiers. Tier 1 contains the functional requirements and the data requirements, because these are the fields the Architecture Agent depends on most heavily; a vague pair of data requirements once produced a broken architecture plan with single-field entities. Tier 2 contains the business goal, user roles, non-functional requirements, acceptance criteria and out-of-scope items, which are asked about when relevant and otherwise defaulted. Tier 3 contains the technology stack, architectural style, constraints and UI and API expectations. API and UI expectations were moved to Tier 3 after feedback that forcing an analyst to list exact endpoints was unnecessary friction: the agent can

infer conventional REST endpoints from the functional and data requirements, and still keeps any the analyst volunteers.

The analyst can also attach a document to a reply. Its full text is read and passed to the gap analysis, capped at 8,000 characters so that a long document cannot silently overflow the model's context in the middle of a field. An earlier reply can be edited, in which case the conversation is rewound to the state just before that reply and regenerated from there.

Before the analyst can confirm, a deterministic quality gate checks the SRS preview. It reports a reason, and blocks confirmation, if there are no functional requirements; if the only functional requirement simply restates the feature description; if an API expectation does not look like a real "METHOD /path" endpoint; if a data requirement reads like a vague sentence rather than a field; if any Tier 1 field was assumed rather than answered; or if the business goal is still the generic template. A data requirement with a field name followed by a type or note, such as "name: string" or "createdAt (set automatically)", is treated as structured however long it is; this rule was added after the gate wrongly flagged a well-annotated field as vague. The gate never prevents a person from confirming, but an override must come with a reason, and that reason is written into the SRS's assumptions so it stays visible to every later reviewer.

4.4.2 SRS structure and generation

The SRS is a JSON document with a fixed set of sections, listed in Table 4.4. Five of them hold items with stable identifiers: functional requirements (FR-001 onwards), non-functional requirements (NFR), acceptance criteria (AC), validation rules (VR) and user stories (US). A traceability section links each functional requirement to its acceptance criteria. The JSON is the form other agents read; a Markdown version and a PDF are generated from it for people.

Table 4.4: Sections of the generated SRS

Group	Sections
Identification	project_id, project_name, project_type, feature_id, feature_name

Group	Sections
Context	target_stack, architectural_style, business_goal, scope, out_of_scope, user_roles
ID-tagged requirements	functional_requirements (FR), non_functional_requirements (NFR), acceptance_criteria (AC), validation_rules (VR), user_stories (US)
Plain-list details	input_requirements, output_requirements, ui_expectations, api_expectations, data_requirements
Risk and context	constraints, assumptions, risks, dependencies
Links	traceability (each FR with its related acceptance criteria)

Generation uses a single schema-constrained call whose prompt tells the model to return JSON only, use stable identifiers, stay within the feature, and record the architectural style only as a preference, since designing the architecture is the next agent's job. The response then passes through a reliability ladder. It is parsed and validated: fourteen required keys must be present, and every functional requirement, non-functional requirement and acceptance criterion must have an identifier and a description. If that fails, a second call asks the model to repair the JSON. If the repair also fails, or the model could not be reached at all, a deterministic fallback builds the SRS directly from the known answers, records the failure reason, and adds a risk entry saying so. The pipeline therefore always produces a document, and the reviewer can always tell how it was produced.

Two deterministic steps run on every SRS regardless of which path produced it. The first normalises plain-list sections that the model sometimes returns in the identifier-and-description shape of the requirement sections. The second is a completeness backstop. The generation prompt did not originally require every section to be non-empty, and the model did return empty user stories in practice. The backstop fills any empty section from content that is already reliable, for example one user story per user role, or constraints derived from the stack and architectural style, and writes a note for each such fill into the assumptions. It never claims an empty risks list means there are no risks; it says the risks were not evaluated.

4.4.3 Revision through targeted operations

When a reviewer asks for a change, the agent does not ask the model to rewrite the SRS. It asks for a short list of operations, each one of add, remove, modify or set,

aimed at a named section. Python then applies them. A new item receives the next free identifier in its section; a modified item keeps its identifier and has its description replaced; a removal matches by identifier, exact text or a distinctive part of the text. If an operation cannot be matched, it is reported back rather than dropped, and malformed operations are skipped rather than allowed to fail the whole revision. After a functional requirement is added or removed, the traceability section is rebuilt so it never points at something that no longer exists.

4.4.4 Knowledge ingestion and retrieval

Figure 4.3 shows the domain pipeline. Ingestion happens once per document. The text is extracted from PDF, Word, text or Markdown files, split into overlapping chunks of 900 characters with 150 characters of overlap, embedded with the all-MiniLM-L6-v2 sentence-transformer model, and stored in a persistent ChromaDB collection. Each chunk's identifier is built from its document and position, so ingesting the same document twice updates the existing vectors instead of duplicating them. Documents uploaded to a project are tagged with that project's identifier so retrieval can be limited to them.

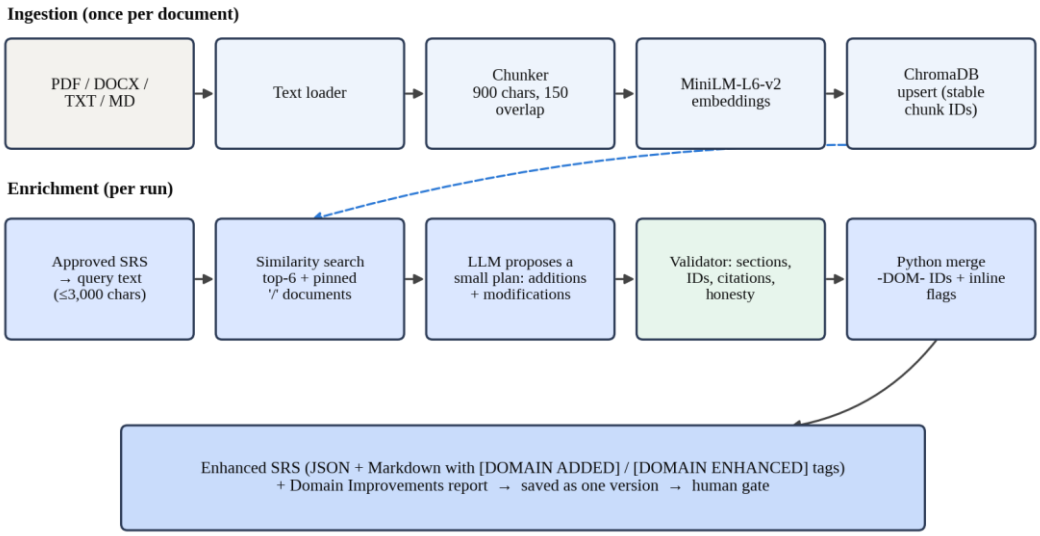


Figure 4.3: Domain knowledge ingestion and the enrichment pipeline

At run time, the Domain Agent builds a retrieval query from the approved SRS. The query uses the descriptive parts of the specification, such as the feature name, business goal and functional requirements, and leaves out sections like constraints and non-functional requirements that would pull retrieval towards generic matches. It

is capped at 3,000 characters. Retrieval returns the six closest chunks. A reviewer can also reference a document explicitly by typing "/" in the chat; every chunk of a referenced document is then included, re-ranked by relevance if the document is large, within caps of six chunks per document, twelve pinned chunks in total and sixteen chunks overall. Retrieval is ordinary Python; the model is not given a search tool.

4.4.5 Enrichment plan, validation and merge

The model receives the SRS and the retrieved chunks, each labelled with its source, and is asked for a plan with two lists. Additions name a target section, a description, a rationale and a citation. Modifications name an existing item's identifier and an enhanced description. The model never assigns identifiers.

The plan is then validated item by item. An addition must target a section that exists; a modification must refer to an identifier that exists. Every item must cite either a chunk that was actually retrieved in this run or, when the reviewer typed a comment, the reviewer's own words, recorded as "human_provided". An honesty check rejects any addition claimed when nothing was retrieved and the reviewer gave no comment. Items that fail are dropped individually with a reason, so one bad guess does not throw away the rest of the plan. If the model's response cannot be used at all, a fallback plan makes no changes and explains why, rather than inventing content.

Finally, Python merges the valid plan into a full copy of the SRS. New items get identifiers with a "-DOM-" marker, such as FR-DOM-001, continuing after any existing ones. A modified item keeps its original description alongside the enhanced one. Every changed item carries its origin and citation, and the Markdown version tags additions as [DOMAIN ADDED] and changes as [DOMAIN ENHANCED]. A separate Domain Improvements report lists what changed, why, and which knowledge sources were used. Because the merge is ordinary code, the structural problems of the old design, such as dropped sections or renumbered identifiers, cannot occur.

Revising an Enhanced SRS needs one more capability: the enrichment plan can only add or enhance, but a reviewer may want something removed. The Domain Agent

therefore reuses the Requirement Agent's revision operations for direct edits, since both documents share the same shape.

4.4.6 Orchestration as a checkpointed state graph

The orchestrator builds one LangGraph state graph, shown in Figure 4.4. Each of the seven stages has a node. The five stages whose output a person must approve before the pipeline continues (requirement, domain, architecture, UI/UX and coder) each have an extra approval node, which calls LangGraph's interrupt function with the feature, the stage and the artefact identifiers to review. The graph stops there. A router after each approval node sends the pipeline to the next stage if the decision was "approved", and back to the same stage's node if it was a rejection or a revision request.

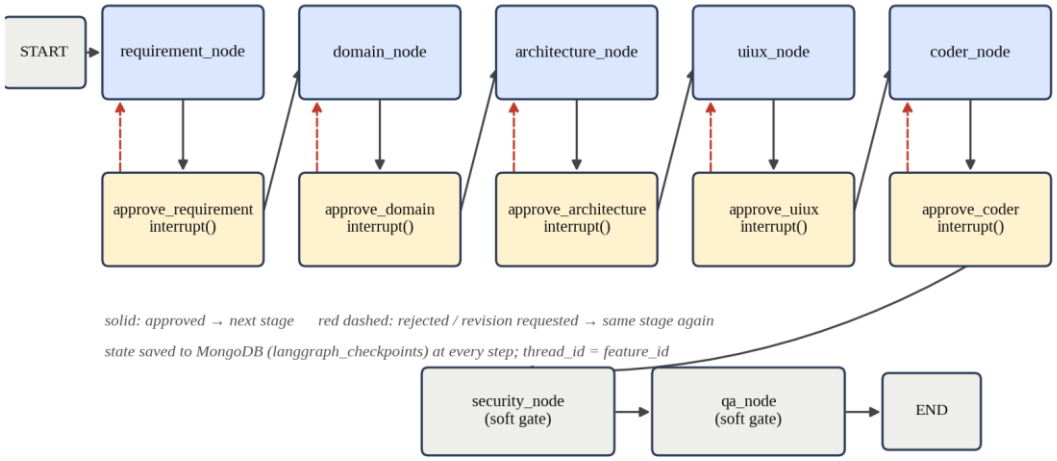


Figure 4.4: The orchestration graph with five hard approvals and two soft gates

The graph is compiled with a MongoDB checkpointer that saves its state to two collections, langgraph_checkpoints and langgraph_checkpoint_writes, after every step. Every call uses the feature's identifier as the thread identifier, so each feature has its own independent pipeline and a decision made hours or days later, by any process that can reach the database, resumes the right one. The orchestrator exposes three operations: start a feature's pipeline, resume it with a decision, and report which node it is waiting at.

The Security and QA stages run as soft gates in the current implementation. They run automatically after the coder stage, and their reports go to a person for review

through the normal approval flow, but the graph does not stop to wait for that decision. A critical security finding is surfaced on the report so the reviewer can send the work back to the Coder Agent.

The requirement node is a pass-through. The Requirement Agent is driven by its own conversation and confirmation endpoints, because elicitation involves many turns with the analyst, not one call. The graph simply waits at the requirement gate until the resulting SRS is approved, and the approval resumes it.

4.4.7 Approval and versioning

Every artefact is stored with its feature, agent, type, format, version number and approval status, which is one of pending, approved, rejected or revision requested. The approval service enforces the rule that nothing moves forward unless it has been approved. When a person approves a gated artefact, the service records the decision and resumes the graph. For the SRS and the Enhanced SRS, only one version may be approved at a time: approving a new version returns the previously approved one to pending. This was added after a user found two SRS versions marked as approved and could not tell which one the Domain Agent would use. An approval can be revoked, but only until the pipeline has moved on to the next stage.

4.5 Standards and professional practice

I followed several established practices in the design. Requirements carry stable, prefixed identifiers and a traceability table, so that later artefacts can refer to them, which is the precondition the traceability literature [6], [7] points to. Acceptance criteria are written in the Given-When-Then form, functional requirements carry a priority such as Must Have or Should Have, and inferred API expectations follow REST conventions.

On the engineering side, all inputs and outputs are defined as typed schemas and validated, deterministic logic is separated from model calls so that it can be tested without a model, and every agent is its own package with its own prompt, schema and modules. Configuration and secrets, such as database connection strings and API keys, are read from environment settings rather than written into the code, and all work was kept under version control in the team's shared repository. Model output is

treated as untrusted input: it is parsed, validated and, where necessary, rejected before it can change a stored artefact.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component is written in Python and runs inside the team's FastAPI backend. Table 5.1 lists the technologies it uses and why each was chosen. Most of the choices were made to keep the system runnable on a single developer machine, without paid services, while leaving the option of a hosted model open.

Table 5.1: Technology stack used by the component

Concern	Technology	Why it was used
Web API	FastAPI, Pydantic	Typed request and response schemas, validation and streaming responses
Orchestration	LangGraph StateGraph	Explicit stage graph with native interrupt and resume
Checkpoints and artefacts	MongoDB (MongoDBSaver)	Already used by the team for artefacts; durable state for the graph
Vector store	ChromaDB (persistent client)	Local, file-backed vector storage with metadata filters
Embeddings	sentence-transformers all-MiniLM-L6-v2	Small, fast, runs locally, independent of the chat model
Chunking	RecursiveCharacterText Splitter	Splits on paragraph and sentence boundaries before characters
Default LLM	qwen3-coder via Ollama	Runs locally and followed the JSON schemas reliably
Optional LLMs	OpenAI and Anthropic APIs	Per-agent override when a stronger hosted model is wanted
Documents	Markdown builder, PDF builder	Human-readable SRS and Enhanced SRS from the same JSON
Frontend	React 19, Vite, Tailwind CSS 4, TanStack Query	The team's shared interface stack
Testing	pytest	Fixture-based tests of the deterministic logic

5.2 Component implementation

Each agent is a self-contained Python package with an entry point, a prompt module, a schema module and its own stage-specific modules. Table 5.2 lists the main modules of my component with their size, which gives a rough idea of where the effort went. The Requirement Agent is the largest because it carries both the conversation and the generation and revision logic.

Table 5.2: Main modules of the component

Module	Lines	Responsibility
requirement_agent/agent.py	1,756	Conversation endpoints, SRS generation ladder, completeness backstop, revision
requirement_agent/conversation_engine.py	541	Gap analysis, tiered questions, known-answer merging
requirement_agent/prompt.py	431	Gap-analysis, generation, repair and revision prompts
requirement_agent/revision_patcher.py	333	Applies add, remove, modify and set operations
requirement_agent/conversation_quality_gate.py	119	Deterministic readiness checks
domain_agent/agent.py	1,273	Retrieval query, plan call, merge, report, revision
domain_agent/domain_validator.py	488	Section, identifier, citation and honesty checks
rag/ (four modules)	364	Loading, chunking, embedding and the ChromaDB store
graph_orchestrator	335	State graph, approval nodes, router, checkpointer
approval_service	563	Decisions, version exclusivity, revocation, resuming the graph
llm_provider_service	384	Provider abstraction and per-agent model selection

5.2.1 Requirement Agent

The conversation is exposed through endpoints to start a conversation, send a reply, stream a reply, edit an earlier reply, and confirm. Each turn calls the gap-analysis prompt and merges the extracted answers into the stored conversation. The streaming endpoint sends partial output back to the interface while the model is still working, which made a noticeable difference to how responsive the chat felt with a local model.

The quality gate is a single function that takes the SRS preview and the set of assumed fields and returns a list of reasons. Keeping it free of model calls meant it could be tested exhaustively and changed safely. The rule that was hardest to get right was the one for vague data requirements. The first version flagged any data requirement longer than six words, which wrongly blocked a well-written field such as "name -- string, required (user's full name or display name)". The final rule first

checks whether the line looks like a field specification, meaning a field name followed by a colon, a dash or a note in brackets, and does not start with a sentence word such as "the" or "when". Only lines that fail this check are judged by their length.

Generation follows the ladder described in Section 4.4.2. In code, each rung is a separate function and each returns either a validated SRS or a reason for failure, so the reason can be carried into the fallback document. The excerpt below shows the shape of the top-level logic, simplified for readability.

```
def generate_srs(answers, feature):
    try:
        raw = llm.invoke(generation_prompt(answers, feature))
        srs, error = parse_and_validate(raw)
        if srs is None:
            fixed = llm.invoke(repair_prompt(raw, error))
            srs, error = parse_and_validate(fixed)
    except LLMUnavailableError as exc:
        srs, error = None, str(exc)
    if srs is None:
        srs = fallback_srs(answers, feature, reason=error)
    srs = normalise_plain_lists(srs)
    return ensure_srs_completeness(srs)
```

The revision patcher works on twelve plain-list sections, the four identifier sections that can be edited directly (FR, NFR, AC and VR), the user stories, and three scalar fields: the business goal, the architectural style and the target stack. Matching for removal and modification tries the identifier first, then the exact text, then a distinctive substring, because reviewers rarely quote a requirement word for word.

5.2.2 Retrieval pipeline

The retrieval code is split into four small modules: a loader that dispatches on file extension, a chunker, an embedder, and a store wrapper around ChromaDB. The embedding model is loaded lazily on first use, so starting the backend does not wait for it, and the embeddings are normalised before they are stored. Knowledge documents are uploaded through an endpoint that accepts files of up to 20 MB, either into the shared knowledge base or into a single project's space.

5.2.3 Domain Agent

The Domain Agent's run function builds the retrieval query, collects the retrieved and pinned chunks, calls the model for a plan, validates it, merges it, and writes three

artefacts: the Enhanced SRS as JSON and Markdown (with a PDF on request) and the Domain Improvements report. The validator is the second-largest module of the agent. Its output is a filtered plan plus a list of dropped items, each with a reason, and the dropped items are logged so that a developer can see what the model tried to do.

Getting the citation rule right took more than one attempt. A citation must name a source document that was really among the chunks retrieved in this run, or the special source "human_provided" when the reviewer typed a comment. In testing, the local model sometimes proposed a change the reviewer had clearly asked for but left the citation empty. Dropping such a change would have ignored the reviewer, so when a comment exists, an item with no citation at all is attributed to that comment instead of being discarded. An item with no citation and no comment is still dropped. Chapter 6 shows that this leniency has a cost.

5.2.4 Orchestrator and approval service

The orchestrator builds and compiles the graph once and exposes start, resume and get-status functions. The graph and its MongoDB checkpointer are synchronous, while the agents are asynchronous. The routes that drive the graph are therefore plain synchronous handlers, which FastAPI runs in a worker thread, and each node runs its agent to completion inside that thread. If a node fails, for example because the model server is down, the checkpoint stays at that node, so the next resume simply retries it. Calling resume on a feature with no running graph raises a specific error that the API turns into a clear message rather than a server error.

The approval service is where a person's decision becomes pipeline movement. When an artefact is approved it updates the artefact's status, enforces the one-approved-version rule for the SRS and the Enhanced SRS, and resumes the graph. A failure in a downstream agent during that resume is caught and logged rather than returned as an error, because the human's approval itself succeeded and should be recorded as such; the failed stage can be retried.

5.2.5 LLM provider service

All model calls go through a provider service that hides the differences between Ollama, OpenAI and Anthropic. Each agent can be given its own provider and

model, so, for example, the Requirement Agent could run on a hosted model while the Domain Agent stays local. When no override is set, the local default is used. API keys are read from the environment and never stored with the artefacts.

5.3 Integration with the other components

My component connects to the rest of AutoForge in three ways. First, through the artefact store: the Architecture Agent, UI/UX Agent and Coder Agent read the latest approved Enhanced SRS, or the approved SRS when there is none. Applying this rule everywhere stopped a class of bugs in which a downstream agent read a draft version. Second, through the orchestrator: every agent is called by its node in the graph, so adding or reordering a stage is a change in one place. Third, through the frontend, where I built the requirement conversation chat, the Domain Agent chat with its document picker, the knowledge panel, the Domain Improvements viewer and the SRS output panel.

Integration produced several of the changes already described. The Architecture Agent's broken entity model led to the Tier 1 data requirements and the stricter quality gate. Finding two SRS versions marked as approved, with no way to tell which one the Domain Agent would use, led to the single-approved-version rule. An early version of the graph approved the domain stage automatically; I moved it back to a hard gate because the Enhanced SRS is exactly the kind of artefact a person should check before design starts.

5.4 Verification and software testing

Testing focused on the deterministic parts of the component, because those are the parts whose behaviour the rest of the system relies on. Table 5.3 summarises the test files and the number of test functions in each; Appendix A gives the full catalogue.

Table 5.3: Automated tests for the component

Area	Test files	Test functions
Requirement Agent	conversation, revision patcher, completeness, normalisation, PDF builder	65

Area	Test files	Test functions
Domain Agent and RAG	fallback, validator, chunking, Markdown builder, PDF builder, retrieval query	42
Orchestration and approval	graph orchestrator, SRS exclusivity, revoke	25
LLM provider	Ollama provider	2
Total	**15 files**	**134**

Some tests check behaviour that matters most to the pipeline. The orchestrator tests confirm that the graph can be resumed by a new orchestrator instance, which simulates a restart; that the domain stage cannot proceed without an approved SRS; and that a rejection loops back to the same stage. The validator tests feed in plans with invented identifiers, unknown sections, citations to chunks that were never retrieved and "human_provided" citations with no human comment, and check that each is dropped with the right reason.

The last recorded run of the full backend suite, on 31 August 2026, collected 1,310 tests across the team's code, of which 160 belong to my component when parameterised cases are counted. The only failure recorded for my component in that run's cache is for a test that has since been replaced, so no current test of the component is recorded as failing. Some tests need a running MongoDB database, because the shared test setup creates a user through the authentication service, which is a limitation I return to in Chapter 7.

The parts that call a model were also checked by hand at each milestone against real feature data, since an SRS can pass every schema check and still say the wrong thing. Several of the issues reported in Chapter 6, such as the irrelevant guest-checkout constraint, were found this way rather than by an automated test.

5.5 Deployment and operation

The component is deployed as part of the AutoForge backend. Running it needs a MongoDB database, an Ollama server with the default model pulled, and a directory for the ChromaDB files. Connection strings, model names and API keys are set through environment variables. The knowledge base is loaded once through the upload endpoint and persists across restarts. Because both the graph checkpoints and

the artefacts live in MongoDB, the backend process can be stopped and started at any time; features that were waiting for a decision are still waiting when it comes back.

6 RESULTS AND VALIDATION

This chapter reports the results in the order of the research questions, after first summarizing what the automated tests established. Pipeline-level numbers come from the team's evaluation. I state where a result belongs to the whole pipeline rather than to my component alone. Section 6.4 then walks through a real run of my two agents on a retail store feature, including the parts that did not go well.

6.1 Research results

Table 6.1 lists the behaviours that matter most to the rest of the pipeline and the tests that check them. All of these were passing in the last recorded run. They do not measure quality; they establish that the deterministic rules the design depends on actually hold.

Table 6.1: Key behaviours confirmed by automated tests

Behaviour	Test file	Result
Vague or assumed Tier 1 fields block confirmation; override needs a reason	requirement_conversation	Pass
Add, remove, modify and set change only the targeted item; unmatched requests are reported	requirement_revision_patcher	Pass
Empty sections are filled from reliable content and noted in the assumptions	srs_completeness	Pass
Invented identifiers, unknown sections and unretrieved citations are dropped	domain_validator	Pass
"human_provided" is rejected when no human comment exists	domain_validator	Pass
An unusable model response gives a no-change plan with a reason	domain_agent_fallback	Pass
Chunk identifiers are deterministic and tagged with their source document	domain_chunking	Pass
A new orchestrator instance resumes a paused feature	graph_orchestrator	Pass
The domain stage cannot run without an approved SRS	graph_orchestrator	Pass
A rejection loops back to the same stage	graph_orchestrator	Pass
Only one SRS version is approved at a time	approval_srs_exclusivity	Pass

Behaviour	Test file	Result
Revoking an SRS is refused once an Enhanced SRS exists	approval_revoke	Pass

6.2 Product and technical results

6.2.1 Effect of retrieval on completeness (RQ1)

The same requirement request was run through the domain stage with retrieval enabled and disabled, using the same model configuration. Table 6.2 and Figure 6.1 show the results.

Table 6.2: Retrieval ablation on one controlled request

Measure	Retrieval ON	Retrieval OFF	Change
Requirements generated	18	15	+3
Useful domain requirements	14	8	+6
Missing requirements	2	7	-5
Human revisions	3	6	-3

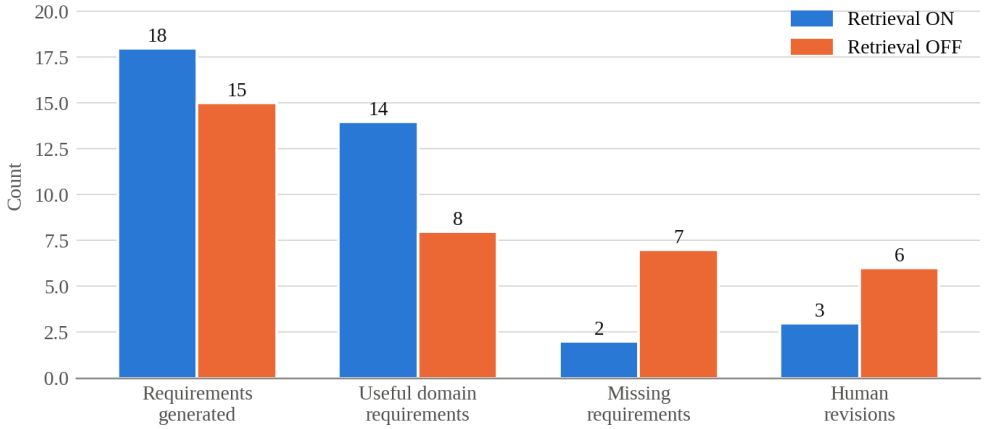


Figure 6.1: Retrieval ON versus OFF on the same request and model

The headline is not the three extra requirements; a longer specification is not automatically a better one. What matters is that the number of useful domain requirements nearly doubled while the number of missing requirements fell from seven to two, and the reviewer needed half as many revisions to reach an acceptable specification. Without retrieval, the model still produced a plausible-looking document, but it missed conventions that the knowledge base states plainly. This is the gap retrieval was meant to close, and on this request it did.

Two cautions apply. This is one request, so the numbers show an effect but not its typical size. And "useful" was judged by the team, not by an independent panel. Section 6.4 also shows that retrieval can add things that are not useful at all.

6.2.2 Small plan versus full regeneration (RQ2)

This question was answered during development rather than in a single experiment. Table 6.3 compares what was observed with the original design, in which the model returned the whole enhanced specification, and with the final design.

Table 6.3: Observed behaviour of the two Domain Agent designs

Property	**Full regeneration**	**Small plan + Python merge**
Models tried	qwen3-coder, gemma4, llama3	qwen3-coder (default)
Required sections	Sometimes silently dropped	Always present; copied by code
Top-level keys	Sometimes missing	Always present
Long outputs	Truncated or malformed JSON	Plan is short; nothing to truncate
Identifiers	Could be renumbered or invented	Never assigned by the model
Uncited additions	Not detectable	Dropped by the validator with a reason
Bad model output	Whole run fails	Bad items dropped; total failure gives a no-change plan

The failures of the first design appeared with all three models, which is why I concluded that the task itself was the problem. With the final design, every Enhanced SRS produced in our runs kept every section and identifier of the approved SRS, because the merge code makes any other outcome impossible. This is a structural guarantee rather than a measured rate. What the small plan does not guarantee is that each addition is relevant, which Section 6.4 illustrates.

6.2.3 Checkpoint recovery (RQ3)

During development, the backend failed unexpectedly in the middle of a pipeline run. After the restart, the pipeline continued from its saved checkpoint instead of starting again. Because one incident cannot give a rate, the team then interrupted the

pipeline deliberately at each of the seven stage boundaries, after the state had been saved. Table 6.4 shows the results.

Table 6.4: Controlled checkpoint-recovery trials

Trial	Interrupted after	Expected next stage	Actual next stage	Correct
1	Requirement	Domain	Domain	Yes
2	Domain	Architecture	Architecture	Yes
3	Architecture	UI/UX	UI/UX	Yes
4	UI/UX	Coder	Coder	Yes
5	Coder	Security	Security	Yes
6	Security	QA	QA	Yes
7	QA	Completion	Completion	Yes

All seven trials resumed at the correct stage without losing the earlier approval decision, a recovery rate of 7 out of 7. Together with the automated test in which a new orchestrator instance resumes a paused feature, this answers RQ3 positively for the conditions tested: process restarts with the database still available. A failure of the database itself was not tested.

6.2.4 Human review at the design stages (RQ4)

In the Item Listing case study, the reviewer did not accept the first SRS. They asked for two user stories for the Admin and Manager roles, which the first version had left out, and the SRS was approved at version 2. The Enhanced SRS was then approved at version 1 without revision. The missing stories matter because they decide which screens and permissions exist. Had they been discovered after the Coder stage, the fix would have touched the architecture plan, the screen designs and the code. Caught at the requirement gate, it was a two-line change.

Across the pipeline, four of the seven stages of Item Listing needed a revision. The team also ran the pipeline with human gates switched on and off, with the results in Table 6.5. These are whole-pipeline results, so they speak to the value of gating in general rather than to my stages alone.

Table 6.5: Pipeline with human gates on and off

Measure	Gates ON	Gates OFF
Successful pipelines	3/3	2/3

Measure	Gates ON	Gates OFF
Average revisions	5.0	1.0
QA pass rate	91%	78%
Security issues	1	4
Escaped defects	1	5
Human review effort	36 min	0

With gates on, more revisions were made, but fewer defects escaped and the QA pass rate was higher. The cost was about 36 minutes of review effort. For RQ4, the case-study evidence shows that review at the requirement stage did catch a real omission before any design existed.

6.3 User or stakeholder validation

No external users or industry stakeholders took part in the evaluation. All reviewer decisions were made within the project team, which knows the system well and may review differently from a business analyst seeing it for the first time. This is the most important gap in the validation of my component, because the value of a conversation and a quality gate depends on how real analysts respond to them.

Informal feedback inside the team did shape the design. Moving API and UI expectations from the required questions to the optional tier, accepting attached documents as answers, and allowing an earlier reply to be edited all came from team members using the requirement chat on their own features.

The panel gave overall positive feedback on the project. It did not comment specifically on the architecture plans, diagrams or page designs.

6.4 Worked Example: Retail Store "Add & List Items"

To show how the component behaves on a real feature, rather than in summary numbers, this section follows the "Add & List Items" feature of a retail store project through the requirement and domain stages. All of the artefacts described here are stored in the project's output folder.

6.4.1 The generated SRS

Table 6.6 summarizes the first SRS produced after the conversation.

Table 6.6: Composition of the retail store SRS, version 1

Section	Count	Notes
Functional requirements	2	Add an item; list items
Non-functional requirements	3	
Acceptance criteria	5	Given-When-Then; includes a rejected image upload case
Validation rules	4	
User stories	2	
User roles	1	"any user"
Data requirements	6	Structured field specifications
API expectations	2	POST /api/items and GET /api/items
Out of scope	3	
Traceability rows	2	One per functional requirement

The SRS is small, which is right for a small feature. Two details show the honesty rules working. The assumptions section states that the UI and API expectations were inferred rather than given, so a reviewer knows those two endpoints were the agent's suggestion. And the single user role, "any user", is exactly the kind of thing a reviewer should question; in the Item Listing case study, the same weakness is what led to the Admin and Manager stories being added.

6.4.2 The Enhanced SRS and its revisions

The Domain Agent produced four versions of the Enhanced SRS. Table 6.7 summarises them.

Table 6.7: Enhanced SRS versions for the retail store feature

Version	Sources used	Changes	Assessment
v1	Product catalogue (2 chunks), checkout and cart (2 chunks)	FR-DOM-001 view item details; two constraints (multiple currencies, guest checkout); AC-003 error message made specific	Detail view and AC-003 useful; guest checkout irrelevant
v2	None	No changes	Correct: the comment asked for nothing concrete
v3	"human_provided"	GDPR constraint added	Citation not justified: the reviewer asked a question
v4	"human_provided"	Same GDPR constraint	Repeat of v3

Version 1 shows both sides of retrieval. Adding a requirement to view an item's details, cited to the product catalogue document, is a sensible addition for a listing feature, and making the upload error message specific improves an acceptance criterion a tester will have to check. But a constraint that the system must support guest checkout has nothing to do with adding and listing items. It was retrieved because the checkout document is close to the SRS in wording, and it passed validation because it cited a chunk that was genuinely retrieved. The validator checks that a citation is real; it cannot check that the cited passage is relevant. A reviewer would need to remove it.

Version 2 is the system behaving as intended. The reviewer's comment contained no concrete request, and the agent made no changes and said so in the report, rather than inventing something to show activity.

Versions 3 and 4 expose a weakness I did not anticipate. The reviewer asked whether any compliance requirements were missing. The model answered by adding a general GDPR constraint and cited the reviewer's comment as its source, with an empty rationale. The validator accepted it, because a human comment existed and the rule only asks whether one exists, not whether it actually requested the addition. In effect, a question was treated as an instruction. The constraint may even be reasonable, but it is not grounded in the knowledge base or in anything the reviewer

stated, which is exactly what the citation rule is meant to prevent. I discuss the fix in Chapter 7.

The approval status of each version is recorded in the artefact store, but I have not relied on it here, because the point of the example is what the agent produced and what a reviewer would need to catch.

6.5 Integration and Performance Results

My component was exercised in every pipeline run, since every feature passes through the requirement and domain stages and is driven by the orchestrator. In the three-feature evaluation, all three features completed all seven stages with gates on. Table 6.8 shows the execution cost of each run as recorded in the execution logs.

Table 6.8: Execution performance of the full pipeline

Run	Total time (min)	LLM calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline	31.2	18	3	25,400

These figures cover all seven stages, and the logs do not separate out the time spent in my stages, so I do not claim a per-stage figure. The number of LLM calls and retries grows with feature complexity, which is what I would expect from my stages too: a more complex feature needs more conversation turns before the quality gate is satisfied, and a longer SRS gives the model more chances to produce invalid JSON. The point that matters for the design is that retries did not stop any run, which is the purpose of the repair-and-fallback ladder.

6.6 Summary of Findings

Table 6.9: Summary of findings by research question

RQ	Finding	Strength of evidence
RQ1	Retrieval raised useful domain requirements from 8 to 14 and cut missing ones from 7 to 2, but can also add irrelevant items	One controlled request plus a real run
RQ2	Small plan and Python merge preserved structure and identifiers in every run; full regeneration failed with three models	Structural guarantee; repeated development observations
RQ3	Seven of seven controlled interruptions resumed correctly with approvals intact	Seven trials, one real failure, automated test
RQ4	The requirement gate caught missing Admin and Manager stories before design; gating reduced escaped defects pipeline-wide	One case study; one on/off comparison

7 DISCUSSIONS

7.1 Interpretation of findings

The results support the main idea behind the component: let the model write, but let deterministic code and a person decide what moves forward. Each research question touched a different part of that idea.

Retrieval helped, but it is not a filter. On the controlled request, it nearly doubled the useful domain requirements and cut the missing ones by more than two thirds. On the retail store feature, it also brought in a guest-checkout constraint that had nothing to do with listing items. Both results come from the same mechanism. Similarity search finds passages that are close to the specification in wording, and most of the time closeness means relevance, but not always. The citation makes the irrelevant addition easy to spot and trace back to its source, which is better than an uncited one, but it still needs a person to remove it.

The small-plan design solved the problem it was built for. Structural failures, such as dropped sections or changed identifiers, cannot happen once the copying is done by code. The lesson I take from it is more general than the Domain Agent: whenever a model is asked to reproduce a large document in order to change a small part of it, the reproduction is the risky part, and it is better to ask only for the change. The same reasoning carried over directly to SRS revision.

Checkpointing behaved exactly as designed, with seven correct recoveries out of seven. This is less surprising than the other results, since persistence and resumption are well-supported by LangGraph and MongoDB. The value of the result is that it removes a practical objection to human gates: a gate that might be lost on a restart is a gate people will want to switch off.

Finally, the early-stage review did catch real problems. The missing Admin and Manager stories and the irrelevant retail store constraint are both cases where a few minutes of reading prevented work that would otherwise have been built on a wrong assumption.

7.2 Comparison with literature baselines and existing solutions

Compared with MetaGPT [2], which also produces an SRS-like document, my component differs in three ways: it asks the analyst before it assumes, it grounds additions in retrieved and cited knowledge, and it holds the pipeline until a person approves the document. Compared with ChatDev [1], the requirement is an artefact with its own versions and decisions rather than a result of agent dialogue. HULA [5] found that engineers value reviewing an agent's plan before code is written; my component applies the same principle one step earlier, to the specification itself, and makes the pause survive a restart.

The use of retrieval follows the idea set out by Lewis et al. [3], but applies it to a different output. Instead of grounding an answer to a question, it grounds additions to a structured document, and it adds a check that each addition cites something that was actually retrieved. Hemmat et al. [4] identified domain grounding in LLM-based requirements engineering as under-explored, and the retrieval ablation is a small piece of evidence that it is worth exploring further.

On traceability, the component does not build the full chain from requirement to code that Gotel and Finkelstein [6] describe. What it does provide is the precondition: identifiers that stay stable through enrichment and revision, and a traceability table linking each functional requirement to its acceptance criteria. Without that, the narrow, single-link solutions that Mucha et al. [7] observe would be the best the pipeline could offer.

7.3 Implications for Practice

For teams building agent pipelines, three practical points follow from this work. First, keep a deterministic check between the model and anything that is stored or passed on. The quality gate, the plan validator and the revision patcher are all small pieces of ordinary code, and between them they prevented most of the failures we saw in early versions. Second, record assumptions where the reviewer will see them. A specification that says "these endpoints were inferred" invites the right question; one that just lists the endpoints does not. Third, make the reviewer's job easier, not just possible. Flags such as [DOMAIN ADDED], a separate report of what changed,

and a quality gate that names the problem all reduce the effort of a careful review, which matters because a gate is only as good as the reading it receives.

7.4 Threats to Validity

Internal validity. The retrieval ablation used one request, and the judgement of which requirements were useful or missing was made by the team. Different reviewers might have drawn the line differently. The human-gates comparison used the same team as reviewers, who knew the system and the expected outcome.

External validity. The knowledge base is five short documents written for the project, and the features are typical e-commerce features. A larger, messier knowledge base, or a different domain, might retrieve less useful passages more often. The evaluation used one default model, so the results may not hold for every model the provider service supports.

Construct validity. Counts of requirements and revisions are a rough measure of specification quality. A specification with fewer missing requirements is better, but the count does not show how serious each missing requirement was. The execution logs also do not separate the cost of my stages from the rest of the pipeline.

7.5 Limitations

Several limitations are specific to my component.

- The citation rule checks that a citation is real, not that the cited passage supports the addition. The guest-checkout constraint passed for this reason.
- The "human_provided" rule only checks that a reviewer comment exists. A question was accepted as the source of a GDPR constraint, and the lenient handling of missing citations makes this easier. A stricter rule would require the comment to contain an explicit request, or would show such additions to the reviewer as suggestions rather than merging them.
- Security and QA run as soft gates in the current implementation. The reports still reach a person, but the graph does not wait for them. We should resolve this

difference, either by making these two stages hard gates again or by describing them as soft gates in future publications.

- Some tests depend on a running MongoDB database through the shared test setup, so the suite cannot run in isolation. The deterministic modules could be tested without it if that setup were separated.
- The quality gate uses fixed rules. They catch the vague inputs we encountered, but a determined or careless analyst can satisfy them with content that is specific in form and still wrong in substance.
- No external analyst used the requirement chat, so its usability for people outside the team is untested.
- A human gate can become a formality if reviewers approve without reading. The system records who approved each artefact, but it cannot tell whether they read it.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value of the Component

The team's common report sets out the commercial case for AutoForge as a whole: teams that build software with AI need to see and approve what the AI produced at each stage. My component sits at the start of that process, where the case is strongest. A wrong or missing requirement costs very little to fix while the specification is being reviewed, but it travels into the design, the code and the tests if nobody catches it. Developers already feel this problem with AI tools. In the 2025 Stack Overflow Developer Survey, 66% named AI output that is "almost right, but not quite" as their biggest frustration, and only 32.7% trusted the accuracy of AI output [23]. An AI that guesses the requirement produces exactly that kind of output.

My component offers three things that answer this. The first is a requirement assistant that asks before it assumes: it fills gaps through a short conversation, flags what it had to assume, and will not let a vague specification be confirmed. The second is domain enrichment that only adds what it can cite. In the retrieval experiment, useful domain requirements rose from 8 to 14, missing requirements fell from 7 to 2, and the reviewer needed half as many revisions (Table 6.2). The third is the orchestration layer, which lets any stage wait for a person and resume after a restart; it resumed correctly in 7 of 7 controlled interruptions (Table 6.4). In the Item Listing case study, the requirement gate caught missing Admin and Manager user stories before any design existed (Section 6.2.4). That is the kind of saving a buyer can understand. Table 8.1 shows who would use the component and what each group gains.

Table 8.1: Target users of the component

User group	Need	What the component provides
Business analysts and project leads in agencies	Turn a short client brief into a complete specification quickly	A guided conversation, a quality gate and an SRS with stable requirement IDs

User group	Need	What the component provides
Developers who write their own requirements	Avoid building the wrong feature	Explicit assumptions and domain additions they can accept or reject
Clients of those agencies	A specification they can read and sign off	A structured SRS and a short report of what the domain stage added and why
Teams in other bounded domains	Domain knowledge they do not have in-house	Knowledge packs for domains such as learning management or hotel booking, with no code changes
Any team running a chain of AI agents	Human approval points that survive delays and restarts	A checkpointed state graph with durable approval, revision and revoke

This evidence comes from our own study, and no external analyst has yet used the component (Section 6.3). The team is therefore surveying developers, analysts and agency staff, as described in Section 7.1 of the common report.

8.2 Target Users and Market Position

The first market is the one AutoForge targets: small e-commerce development teams and agencies, beginning in Sri Lanka. The country's IT and business process management industry has more than 680 companies and over 700 active startups, employing about 175,000 people [24]. Most of these firms write requirements in some form for every client project, so a tool that shortens that work without lowering its quality has a broad base of potential users.

Table 8.2 compares the component with what these users can do today. The prices are the published team-plan prices in October 2026, converted at 330.43 rupees to the dollar [30].

Table 8.2: Alternatives for producing a feature specification

Alternative	Typical cost	Strength	Weakness compared with this component
Writing the SRS by hand	Analyst time (one or more days per feature)	Full control and judgement	Slow; no domain prompts; quality depends on the writer's experience
General chat assistant (ChatGPT Business, Claude Team)	USD 25 (about LKR 8,260) per user per month [26], [27]	Fast and flexible	Fills gaps silently; no fixed structure or stable IDs; nothing stops a vague draft
Prompt-to-app builders (Lovable, Bolt)	USD 25 to 50 per user per month [28], [29]	Working app in minutes	Skip the specification altogether; nothing for a client to sign off
Role-based frameworks such as MetaGPT [2]	Free (open source)	Specification is part of a larger pipeline	Written from model knowledge only; no human gate or durable pause
This component (within AutoForge)	Included from the free Community edition	Asks for gaps; quality gate; cited enrichment; stable IDs; durable approval	Slower than a single prompt; domain quality depends on the knowledge base

The component is not meant to be sold on its own at first. It is the entry stage of AutoForge, and every feature passes through it. Its competitive position rests on two things that the alternatives in Table 8.2 lack. The first is that it treats an unclear requirement as something to ask about rather than something to guess. The second is that its domain additions come with a citation and a reason, so a reviewer can check them. Both can be copied in principle, but the knowledge packs behind the second take domain expertise to write, which makes them the most defensible part of the component.

8.3 Business Model and Feasibility

The component fits within the open-core model and the packages proposed in the common report (Table 7.5): a free self-hosted Community edition, Team seats at LKR 6,000 per month, Agency seats at LKR 10,000 per month, and enterprise and academic licenses. Table 8.3 shows the parts of that model that depend directly on my work.

Table 8.3: Revenue elements that depend on this component

Revenue element	Price (common report)	What this component contributes
Domain knowledge packs	LKR 30,000 per pack; one included in Agency	The validator and merge contain no domain rules, so a new pack is a set of documents, not new code
Team and Agency review features	LKR 6,000 and 10,000 per seat per month	Approval, revision, revoke and version history already exist in the approval service; roles and an audit log extend it
Enterprise / on-premise licence	From LKR 1,500,000 per year	The orchestrator runs with a local model and a MongoDB checkpointer, so the whole pipeline can stay inside a customer's network

Revenue element	Price (common report)	What this component contributes
Setup and training services	LKR 75,000 per engagement	Building a client-specific knowledge base from the client's own documents

Knowledge packs deserve a closer look, because they are the revenue stream that comes most directly from this component. The e-commerce base used in the evaluation consists of five short documents (Table 3.1). Writing and testing a pack of similar size for a new domain would take an analyst about three to five days. At typical Sri Lankan rates for a software engineer, that is roughly LKR 25,000 to 80,000 of effort. A pack priced at LKR 30,000 therefore recovers its cost after two or three sales, and every later sale is almost pure margin. Before release, each pack would be tested with the retrieval ablation from Chapter 3, so that its effect on missing requirements is measured rather than assumed.

Technically, the component is already feasible: it runs on a single machine with a local model and was used in every feature of the evaluation. The domain stage is also cheaper to run than the original design, because the model returns a short plan instead of retyping the whole specification. The execution logs do not separate the cost of my stages from the rest of the pipeline (Section 6.5), so I rely on the whole-pipeline figure: one feature run costs nothing in API fees with the local model, and at most about LKR 160 on a hosted model. Table 8.4 lists the work that the results in Chapter 6 show is needed before the component could be sold.

Table 8.4: Work needed before a commercial release

Issue found	Evidence	Change needed
An irrelevant constraint was retrieved	Guest-checkout constraint (Section 6.4)	A relevance threshold on retrieved chunks, and packs tested before release

Issue found	Evidence	Change needed
The citation check could be satisfied by any reviewer comment	GDPR constraint cited to a question (Section 6.4)	Check that the comment actually asked for the addition
No external analysts have used the component	Section 6.3	A pilot with one agency's analysts, measuring revisions and review time
Library versions change quickly	LangGraph, ChromaDB and sentence-transformers	Pinned versions and planned upgrades

8.4 Sustainability and Maintenance

Technical sustainability depends mostly on how well the component copes with change. Three design choices help. The provider service means a better or cheaper model can be adopted by configuration, not by rewriting agents. The deterministic gate, validator and merge code do not depend on the model, so they keep working when the model changes. And the knowledge base can be updated by uploading documents, without retraining anything.

There are also maintenance risks. The component depends on LangGraph, ChromaDB and sentence-transformers, all of which change quickly, so their versions need to be pinned and upgraded deliberately. The quality gate's rules were tuned on our own features and will need review as the component meets new kinds of requirement. And knowledge packs age: payment and data protection rules in particular change, so each pack needs an owner and a review date.

Socially, the component keeps a person in the role of deciding what the software should do. It is designed to make an analyst faster and more thorough, not to replace them, and the conversation format means the analyst is involved in every answer.

The main ethical risk is the one discussed in Chapter 7: if reviewers start approving specifications without reading them, the gate stops protecting anything. Clear flags, visible assumptions and a short Domain Improvements report are meant to keep review practical.

8.5 Contribution to the Sustainable Development Goals

The common report links AutoForge to SDG 9 (Industry, Innovation and Infrastructure) and SDG 8 (Decent Work and Economic Growth). My component's contribution to SDG 9, and in particular target 9.5 on upgrading technological capabilities, is a method for using AI in the earliest, most decision-heavy stage of software development while keeping a person accountable, with a model that can run locally rather than through an expensive external service. Its contribution to SDG 8, targets 8.2 and 8.3, would come through small development firms, which could produce clearer specifications for their clients in less time and spend less on rework.

These contributions are potential rather than demonstrated, since the component has not been used by a real business. There are also possible negative effects. If analysts start approving specifications without reading them, the gate stops protecting anything and their own skills may weaken. The component tries to prevent this by keeping the analyst involved in every answer of the conversation, showing assumptions and domain additions openly, and keeping the Domain Improvements report short enough to read. The next step responsible would be a pilot with one small agency, measuring the same things as in Chapter 6, together with the time analysts spend in the requirement conversation and the energy used by the local model.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to make the first stages of an LLM-based development pipeline produce a specification that is complete, grounded, safely revisable and approved by a person, inside a pipeline that can wait for that approval as long as it takes. The component I built does this through three ideas used together: ask the analyst instead of guessing, ask the model only for small changes and let code apply them, and stop the pipeline at a durable checkpoint until a person decides.

The evaluation showed that each idea did its job. Retrieval with cited enrichment produced a more complete specification on a controlled request. The small-plan design removed the structural failures that had made the first Domain Agent unusable with three different models. The checkpointed state graph resumed correctly in every controlled interruption. And review at the requirement stage caught an omission that would otherwise have spread into the design and the code. The evaluation also showed where the design is weak: retrieval can bring in irrelevant content with a perfectly valid citation, and the rule for citing a reviewer's comment can be satisfied by a question. Neither weakness would have been visible without a person reading the output, which is the strongest argument for keeping that person in the loop.

9.2 Achievement of objectives

Table 9.1: Achievement of the objectives

Objective	Outcome	Status
O1: Requirement Agent with conversation, quality gate, schema-valid SRS and stable IDs	Implemented with three-tier questions, deterministic gate with recorded overrides, repair-and-fallback ladder, completeness backstop and targeted revision	Achieved
O2: Retrieval and Domain Agent that only add cited knowledge without regeneration	Implemented with ChromaDB retrieval, small plan, validator and Python merge; citation relevance not yet checked	Largely achieved

Objective	Outcome	Status
O3: Orchestration that pauses at gates and resumes after any delay or restart	Implemented as a LangGraph state graph with MongoDB checkpoints; Security and QA run as soft gates	Achieved
O4: Evaluate retrieval, recovery and early-stage review	Retrieval ablation, seven recovery trials, case-study review records; small scale, no external users	Achieved at limited scale

9.3 Contribution

The main contribution of this work is a tested design for the specification stages of an agent pipeline, in which the model proposes and deterministic code and a person dispose. More specifically, it contributes a conversational requirement agent with a rule-based readiness gate; a "small plan, deterministic merge" pattern for enriching and revising a structured document, together with evidence of why the alternative fails; a citation and honesty validator for retrieval-grounded additions; and a checkpointed orchestration layer that makes human gates practical across restarts. It also contributes an honest account of two failure modes, irrelevant but validly cited retrieval and question-as-instruction citations, which other work on retrieval-grounded requirements can test against.

9.4 Future directions

The most direct follow-up is to close the two weaknesses found in Chapter 6. A relevance check could compare each addition with the feature's functional requirements and mark weak matches for the reviewer instead of merging them silently. The "human_provided" rule could require that the reviewer's comment contains a request rather than a question, and additions attributed to a comment could be shown as suggestions for the reviewer to accept.

Beyond that, the evaluation should be repeated at a larger scale: more requests in the retrieval ablation, more than one model, and, most importantly, real business analysts using the requirement conversation on their own features. The knowledge base should be extended, and the same evaluation run on a second domain to test the claim that the design is domain-independent. On the orchestration side, the difference between the implementation and the paper over the Security and QA gates should be

settled, ideally by letting a team choose which stages are hard gates, since our results suggest that some stages need review more than others. Finally, the traceability table could be extended forward, so that each requirement links to the architecture elements, screens, code changes and tests that implement it, completing the chain that stable identifiers make possible.

REFERENCES

[1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 23247–23275.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[4] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: A systematic review," *Frontiers in Computer Science*, vol. 7, Art. no. 1519437, 2025.

[5] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025, pp. 342–352.

[6] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. IEEE Int. Conf. Requirements Engineering*, 1994, pp. 94–101.

[7] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.

[8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.

[9] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025.

[10] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.

[11] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, “When software security meets large language models: A survey,” *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[12] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528–50652.

[13] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, “OpenHands: An open platform for AI software developers as generalist agents,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, pp. 65882–65919.

[14] Chroma, “Chroma documentation.” [Online]. Available: https://docs.trychroma.com/ (accessed Aug. 25, 2026).

[15] K. Pelluru, “LangChain & LangGraph in production: Architectures for multi-agent LLM systems,” *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[17] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” *arXiv preprint* arXiv:2402.01030, 2024.

[18] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, “HyperAgent: Generalist software engineering agents to solve coding tasks at scale,” *arXiv preprint* arXiv:2409.16299, 2024.

[19] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” *arXiv preprint* arXiv:2407.01489, 2024.

[20] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 54107–54157.

[21] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “AgentCoder: Multi-agent-based code generation with iterative testing and optimisation,” *arXiv preprint* arXiv:2312.13010, 2023.

[22] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 2471–2484.

[23] Stack Overflow, “2025 Developer Survey: AI,” 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai (accessed Oct. 8, 2026).

[24] “SLASSCOM reflects on a year of creating opportunities, strengthening Sri Lanka’s knowledge and innovation industry,” Daily FT, Jul. 15, 2026. [Online]. Available: https://www.ft.lk/it-telecom-tech/SLASSCOM-reflects-on-a-year-of-creating-opportunities-strengthening-Sri-Lanka-s-knowledge-and-innovation-industry/50-794672 (accessed Oct. 8, 2026).

[25] “Data protection compliance regime takes effect on 1 Jan. 2027,” Daily FT, 2026. [Online]. Available: https://www.ft.lk/front-page/Data-protection-compliance-regime-takes-effect-on-1-Jan-2027/44-795776 (accessed Oct. 8, 2026).

[26] “ChatGPT pricing in 2026: Free, Go, Plus, Pro, Business and Enterprise,” GPTPrompts.ai. [Online]. Available: https://gptprompts.ai/ai-pricing/chatgpt-pricing (accessed Oct. 8, 2026).

[27] Anthropic, “Claude pricing.” [Online]. Available: https://claude.com/pricing (accessed Oct. 8, 2026).

[28] Lovable, “Subscription plans.” [Online]. Available: https://docs.lovable.dev/introduction/subscription-plans (accessed Oct. 8, 2026).

[29] StackBlitz, “Bolt pricing.” [Online]. Available: https://bolt.new/pricing (accessed Oct. 8, 2026).

[30] Exchange-Rates.org, “USD to LKR: Convert US Dollars to Sri Lanka Rupees.” [Online]. Available: https://www.exchange-rates.org/converter/usd-lkr (accessed Oct. 7, 2026).

GLOSSARY

Acceptance criterion	A testable condition, written here as Given-When-Then, that a feature must meet to be accepted.
Artefact	A versioned output of a pipeline stage, such as an SRS, stored with its approval status.
Checkpoint	The saved state of the pipeline graph, from which it can resume after a pause or restart.
Chunk	A short, overlapping piece of a knowledge document that is embedded and retrieved as a unit.
Deterministic fallback	Code that produces a usable output without a model when the model's output cannot be used.
Embedding	A vector of numbers representing the meaning of a piece of text, used to find similar text.
Enhanced SRS	The SRS after the Domain Agent has added or enriched items, each flagged and cited.
Gap analysis	The per-turn model call that works out what is still missing from the requirement.
Hard gate	A stage at which the pipeline stops until a person approves, rejects or requests a revision.
Human-in-the-loop	A design in which a person makes decisions inside an automated process, not only at the end.
Interrupt	LangGraph's mechanism for pausing a graph at a node until it is resumed with a value.
Quality gate	The rule-based check that blocks confirmation of a vague or assumption-heavy SRS.
Retrieval-augmented generation	Generating text with relevant retrieved passages supplied to the model as context.

APPENDICES

Appendix A Supporting Technical Material

A.1 Test catalogue

Table A.1: Test files for the component

Test file	Functions	What it covers
test_requirement_conversation	19	Conversation turns, tiers, editing replies, quality gate and overrides
test_requirement_revision_patcher	21	Add, remove, modify and set; matching; unmatched reporting; traceability rebuild
test_srs_completeness	11	Backstop filling of empty sections and assumption notes
test_srs_normalization	6	Plain-list normalisation of model output
test_requirement_pdf_builder	8	SRS PDF generation
test_domain_agent_fallback	10	No-change fallback plan, merge, DOM identifiers, metadata
test_domain_validator	10	Section, identifier, citation and honesty checks
test_domain_chunking	5	Chunk sizes, empty input, deterministic identifiers
test_domain_markdown_builder	5	[DOMAIN ADDED] and [DOMAIN ENHANCED] Markdown output
test_domain_pdf_builder	8	Enhanced SRS PDF generation
test_domain_retrieval_query	4	Fields included and excluded, revision text, length cap
test_graph_orchestrator	5	First-gate pause, resume after a new instance, gated domain stage, rejection loop
test_approval_srs_exclusivity	6	One approved SRS and Enhanced SRS version at a time
test_approval_revoke	14	Revocation, cascades and blocking once the next stage exists
test_ollama_provider	2	Context window size sent with generate and stream calls
Total	**134**	

A.2 Quality gate rules

Table A.2: Rules applied by the requirement quality gate

Rule	Blocks confirmation when
No functional requirements	The SRS preview has no functional requirements
Restated description	The only functional requirement repeats the feature description
Malformed API expectation	An API expectation is not in the form "METHOD /path"
Vague data requirement	A data requirement is not a field specification and is longer than six words
Assumed Tier 1 field	Functional or data requirements were defaulted rather than answered
Boilerplate business goal	The business goal is still the generic template sentence
Override	Allowed for any rule, but the reason is written into the SRS assumptions

A.3 Domain Improvements report, retail store version 1

The extract below shows one addition from the Domain Improvements report for version 1 of the retail store Enhanced SRS, as stored in JSON. It is the guest-checkout constraint discussed in Section 6.4.

```
{
  "target_section": "constraints",
  "new_id": null,
  "description": "The system must support guest checkout,
    capturing a valid email address and shipping address
    even if the user does not create an account.",
  "rationale": "This is a standard e-commerce requirement
    to allow for flexible customer interactions.",
  "domain_citation": {
    "source_document": "checkout_and_cart_conventions.txt",
    "chunk_id": "checkout_and_cart_conventions.txt#1"
  }
}
```

Appendix B: Disclosure of AI Tool Use

Table B.1: Use of generative AI tools

Activity	Tool	How it was used	How the output was checked
System runtime	qwen3-coder via Ollama	The model inside the Architecture and UI/UX agents, as described in this thesis	Deterministic validation, tests and human review
Figures	Claude (Anthropic)	Drawing the design diagrams.	Checked against the implementation by the author